

claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_887fb14e-e6d\agent-  
a335a4e347b5b0df6.jsonl`  
- SHA-256: `2e168573341b0f65244d54b346bb466d12ad79a27ee088edfec389a783e53bf8`  
- Source modified: `2026-06-20T10:09:59+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_887fb14e-e6d`  
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`
```

Transcript

1. user (2026-06-20T10:03:00.472Z)

Goal: design dedicated Studio-UI pages that turn "annotate your footage ? train your OWN model" into a first-class USP (the bring-your-own-model moat), linking the public GitHub repo (<https://github.com/avidullu/rally-annotator>), with screenshots and step-by-step setup/use help. Be HONEST about what is real today vs roadmap. Tag claims [verified]/[design]/[researched]. Do NOT invent capabilities. EXCLUDE any .claude/worktrees/ paths.

Read the rally-annotator repo at C:\\Users\\avidu\\Projects\\rally-annotator (its own checkout). Read README + source + any docs/images. Map: what it is, install/setup (it is a VLC plugin/extension ? how a user installs it on Win/Mac/Linux), the annotation workflow (mark rally start/end + point-stop reason, the keys/buttons, saving), the exact OUTPUT CSV schema (columns + where written), any screenshots/images present in the repo (give paths; if none, say so), and repo facts (license, latest version, supported sports). Quote path:line. Return the schema.

2. assistant (2026-06-20T10:03:03.929Z)

I'll research the rally-annotator repo to map what it actually is, how it installs, the annotation workflow, output schema, screenshots, and repo facts. Let me start by pulling the repo (per the freshness convention) and exploring its structure.

3. user (2026-06-20T10:03:07.211Z)

From <https://github.com/avidullu/rally-annotator>
b5c51b4..6653d89 claude/intelligent-planck-fokdq2 -> origin/claude/intelligent-planck-fokdq2
Already up to date.

---BRANCH---

main

---LOG---

d5d59f2 v1.6.4: guard unsaved rallies from silent loss + flag edit mode on Mark buttons (#13)
6653d89 v1.6.4: guard unsaved rallies from silent loss; flag edit mode on Mark buttons
0262ed5 v1.6.3: show the version next to the plugin in VLC's Active Extensions list (#12)
9373703 v1.6.3: show the version next to the plugin in VLC's Active Extensions list
4808a3a v1.6.2: adopt the video's CSV lazily (fix resume saving to home fallback) (#11)

4. user (2026-06-20T10:03:07.310Z)

```
C:\Users\avidu\Projects\rally-annotator\LICENSE  
C:\Users\avidu\Projects\rally-annotator\.gitignore  
C:\Users\avidu\Projects\rally-annotator\.git\description  
C:\Users\avidu\Projects\rally-annotator\.git\hooks\applypatch-msg.sample  
C:\Users\avidu\Projects\rally-annotator\.git\hooks\commit-msg.sample  
C:\Users\avidu\Projects\rally-annotator\.git\hooks\fsmonitor-watchman.sample  
C:\Users\avidu\Projects\rally-annotator\.git\hooks\post-update.sample  
C:\Users\avidu\Projects\rally-annotator\.git\hooks\pre-applypatch.sample  
C:\Users\avidu\Projects\rally-annotator\.git\hooks\pre-commit.sample
```

C:\Users\avidu\Projects\rally-annotator\.git\hooks\pre-merge-commit.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\pre-push.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\pre-rebase.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\pre-receive.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\prepare-commit-msg.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\push-to-checkout.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\sendemail-validate.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\update.sample
C:\Users\avidu\Projects\rally-annotator\.git\info\exclude
C:\Users\avidu\Projects\rally-annotator\.git\objects\06\015abe347817c322586dddea658a8382d9b8d7
C:\Users\avidu\Projects\rally-annotator\.git\objects\36\6885e442e24c433b9511797e62bc5330885f6c
C:\Users\avidu\Projects\rally-annotator\.git\objects\3f\232efa99468a4af899f0ee46e3fc5a78316af0
C:\Users\avidu\Projects\rally-annotator\.git\objects\d6\d193674a3dfb0c58280f295d946c72c5f9d232
C:\Users\avidu\Projects\rally-annotator\.git\objects\fa\bf09962ccf48626095c050d6135e99918a38e1
C:\Users\avidu\Projects\rally-annotator\.git\objects\e1\4e44f360577f5f83c4f3cd991921f63ba3464d
C:\Users\avidu\Projects\rally-annotator\.git\objects\f9\8a7512639a022203551ade1537a6f4fbf2df6e
C:\Users\avidu\Projects\rally-annotator\.git\objects\fa\2a55b1471eb204fdedb4b04a26c1753fdb4807
C:\Users\avidu\Projects\rally-annotator\.git\objects\50\7909c1a94cec6261099991be5a63b7e9e0f6a6
C:\Users\avidu\Projects\rally-annotator\.git\objects\7c\10778e78cae28070295d2c612c332f6e67e0e2
C:\Users\avidu\Projects\rally-annotator\.git\objects\8b\60c85dc70a2b8ae5c3f6cb8891402102c7b0f6
C:\Users\avidu\Projects\rally-annotator\.git\objects\b9\63d8b87bf811cfb86d661253acc41f92429462
C:\Users\avidu\Projects\rally-annotator\.git\objects\e9\c4c0f330e832b8ba687ad4c77bee0c9fcaad1f
C:\Users\avidu\Projects\rally-annotator\.git\objects\b7\84ea474c54cf9be75901b06e8b6882e3b0fff2
C:\Users\avidu\Projects\rally-annotator\.git\objects\64\09f2ed9854e9341a826038b65f79fd8a7e58d1
C:\Users\avidu\Projects\rally-annotator\.git\objects\98\d2e7d1815bb210b52e79f9df3d04f8d81ef800
C:\Users\avidu\Projects\rally-annotator\.git\objects\b0\58262152176be49a72dfd55f297376648a50c
C:\Users\avidu\Projects\rally-annotator\.git\objects\d4\c682a8ff2e3271aa60eab596be23279c59f9ae
C:\Users\avidu\Projects\rally-annotator\.git\objects\11\4489abd4a52d837b3067e4d1c54fc0bfa540a1
C:\Users\avidu\Projects\rally-annotator\.git\objects\76\31908f23c077c9420450c0c7267fa30b12507a
C:\Users\avidu\Projects\rally-annotator\.git\objects\30\26bb485e8988caf8ef01159ab3aa4458c06cd5
C:\Users\avidu\Projects\rally-annotator\.git\objects\8d\42a714e8e1f61f68964b29e669a4d5db03ff6f
C:\Users\avidu\Projects\rally-annotator\.git\objects\b6\6154751399ab39937e1761192002da2c7acfb9
C:\Users\avidu\Projects\rally-annotator\.git\objects\20\f14f77a411e808112e3eaf49282707e89c4c69
C:\Users\avidu\Projects\rally-annotator\.git\objects\d2\8c7565a6ce65007b354a573a61d25f5abea149
C:\Users\avidu\Projects\rally-annotator\.git\objects\84\7e7be4c5451f9772b643ebdcf82a70a11d5c1e7
C:\Users\avidu\Projects\rally-annotator\.git\objects\37\463531b4c563410fb5a55ebe9338da2c3c98e8
C:\Users\avidu\Projects\rally-annotator\.git\objects\8c\6a969230193a68d384afcebcbaada5048903f95
C:\Users\avidu\Projects\rally-annotator\.git\objects\cd\bc09d8440d69192c0c316c5521d3829d75084a
C:\Users\avidu\Projects\rally-annotator\.git\objects\d1\9980e1fd79dfd2f5e90a2038de83a5471eeaa0d
C:\Users\avidu\Projects\rally-annotator\.git\objects\42\4eb3ccd5f7bf916f0c912c9595387d9123c92e
C:\Users\avidu\Projects\rally-annotator\.git\objects\60\5bcf6e8ca3c69a9bc61e545a4ce6544008ffc5
C:\Users\avidu\Projects\rally-annotator\.git\objects\b9\77e2d38594c9d5ac50138fe60b9cd9c9e54f0e
C:\Users\avidu\Projects\rally-annotator\.git\objects\10\af409917d144d4c5a29f85400ba5d29c2f190f
C:\Users\avidu\Projects\rally-annotator\.git\objects\37\7a0ec10c6048354d3a0c824d4bb8b1caa907f00
C:\Users\avidu\Projects\rally-annotator\docs\ENDING_REASONS.md
C:\Users\avidu\Projects\rally-annotator\.git\objects\84\c8a8b28b96c2cdedc1fc6378feab9a30313eb8
C:\Users\avidu\Projects\rally-annotator\.git\objects\08\1960fe673318fdb66b340a992f52d718f3f68b
C:\Users\avidu\Projects\rally-annotator\.git\objects\b0\6094ad44ec13677f61429efc19eb943f6c7331
C:\Users\avidu\Projects\rally-annotator\.git\objects\25\298d09c5d34e1058915e6add080fa2a626b0ce
C:\Users\avidu\Projects\rally-annotator\.git\objects\10\7e60616d2679a86ebf33046f1ada31fd3513c7
C:\Users\avidu\Projects\rally-annotator\.git\objects\f2\c743e07bee85896646fba318488a813201824
C:\Users\avidu\Projects\rally-annotator\.git\objects\36\8e4f1a8bbfab167cb92136802217da9668c0fc
C:\Users\avidu\Projects\rally-annotator\.git\objects\38\ef62b7938e0a7ca288e31c20070a233e0e6f8e
C:\Users\avidu\Projects\rally-annotator\.git\objects\fa\7cffffa190358fd587fa1c0d1740fe99ea27d2e
C:\Users\avidu\Projects\rally-annotator\.git\objects\9b\fffa88224f7f90e91ddf2a0b3392a10d8218d45
C:\Users\avidu\Projects\rally-annotator\.git\objects\74\7f0fb41e743fc0c7df80c88b63db1c6f3f580dc
C:\Users\avidu\Projects\rally-annotator\.git\objects\d8\fc66fa92776c30bb95b8ef9a1c8e3fcc211802
C:\Users\avidu\Projects\rally-annotator\.git\objects\e1\0e5438d6c4f261119e85ed450ea8486df29504
C:\Users\avidu\Projects\rally-annotator\.git\objects\3d\5eeb4ff53eb5793c409292f3f6cf171bb3c890
C:\Users\avidu\Projects\rally-annotator\.git\objects\b5\c51b4ef23ed3e0398db8b6633ec5b1731fea9c
C:\Users\avidu\Projects\rally-annotator\.git\objects\02\7cb297e488537cf0c605251e539704bf757ef2
C:\Users\avidu\Projects\rally-annotator\.git\objects\79\bd243ab5aab1cce6a440e1a1c517796d3093e3
C:\Users\avidu\Projects\rally-annotator\.git\objects\aa\b3b7650c31aa10c03f1868d66d8564853d2f99
C:\Users\avidu\Projects\rally-annotator\.git\objects\db\d570dc2b7a5ed387ada9e05a3e79bd997b8410
C:\Users\avidu\Projects\rally-annotator\.git\objects\7b\200d4b2e9eb0073ba2cd5c04b4483c12f50382

```
C:\Users\avidu\Projects\rally-annotator\.git\objects\f9\e34e78824ba67b8ab071666373da5d0035728b
C:\Users\avidu\Projects\rally-annotator\.git\objects\23\ab53cae71147151e67bfa39bd809b44207266b
C:\Users\avidu\Projects\rally-annotator\.git\objects\d3\65b14261f8979c8f2c364efebfe35ed400792d
C:\Users\avidu\Projects\rally-annotator\.git\objects\67\dd08cd318d8215a76546cff5362b819de5b724
C:\Users\avidu\Projects\rally-annotator\.git\objects\9f\1f267890316f6d01f44dc5c7be6c565a3ef4a6
C:\Users\avidu\Projects\rally-annotator\.git\objects\d8\39ad74b2a0ecc98ce9cfe11c1e6a380fff64eb
C:\Users\avidu\Projects\rally-annotator\.git\objects\dd\1c54a5651f3b1acd1be67b4fbd9e248f34576a
C:\Users\avidu\Projects\rally-annotator\.git\objects\4a\92bc6a868619892384f52b083676c9770c681e
C:\Users\avidu\Projects\rally-annotator\.git\objects\df\743c7109ff5c93a39500dbd307bdd1ee97bfb1
C:\Users\avidu\Projects\rally-annotator\.git\objects\ee\86d7d372127fb5b6672931734e3f0c7fea9fb8
C:\Users\avidu\Projects\rally-annotator\.git\objects\ca\ad10fa86b0fe19f3b27538b4a092c2c2e84bf8
C:\Users\avidu\Projects\rally-annotator\.git\objects\5b\0eb2157919533da0eff733c1f8c42601d2ab49
C:\Users\avidu\Projects\rally-annotator\.git\objects\5b\f046f6c4b514935e5906f4b6b9d1a16bf52e98
C:\Users\avidu\Projects\rally-annotator\.git\objects\f2\8735847a915c9efe53b63ec6670b203b637d7e
C:\Users\avidu\Projects\rally-annotator\.git\objects\43\59082343da2a7aac90e3e1ac4e18351d3c4fac
C:\Users\avidu\Projects\rally-annotator\.git\objects\64\2d00619e1e71bed53a7b259eca3af2e189faef
C:\Users\avidu\Projects\rally-annotator\.git\objects\8d\69c13045a99ff9edac22b7dc04c8e4294a0dbd
C:\Users\avidu\Projects\rally-annotator\.git\objects\1c\dfbe89a10848abccf3fc76325bc8aaa193d26b
C:\Users\avidu\Projects\rally-annotator\.git\objects\80\04a236152999ad702fb6957c1b4c0e71d76966
C:\Users\avidu\Projects\rally-annotator\.git\objects\4f\d96d2d38a61f603ef6f417007c4ac951671632
C:\Users\avidu\Projects\rally-annotator\.git\objects\cb\5ed55afb7ef35321e724da30655b09911a573
C:\Users\avidu\Projects\rally-annotator\.git\objects\f7\3abbbe60c437b82f34a2d690b881e4b1011191
C:\Users\avidu\Projects\rally-annotator\.git\objects\d9\c98f9ca5def221b71a02de79bb17b5c37530ee
C:\Users\avidu\Projects\rally-annotator\.git\objects\7d\427ba386c4fd6af0641ea2104a6e86f93a4910
C:\Users\avidu\Projects\rally-annotator\.git\objects\43\5267c08851c082ce142104fa0ede630a452d29
C:\Users\avidu\Projects\rally-annotator\.git\objects\9d\d141346241947603c278c34fc954f5480a782b
(Results are truncated. Consider using a more specific path or pattern.)
```

5. assistant (2026-06-20T10:03:11.166Z)

Let me read the key files: README, source code, docs, and check for images. Let me get a clean (non-.git) file listing first.

6. user (2026-06-20T10:03:12.393Z)

```
.gitattributes
.github/workflows/ci.yml
.gitignore
CHANGELOG.md
LICENSE
README.md
docs/CSV_FORMAT.md
docs/ENDING_REASONS.md
test/README.md
test/dialog_layout.snapshot
test/dialog_test.lua
vlc/rally_annotator.lua
```

7. assistant (2026-06-20T10:03:14.219Z)

A clean, focused repo. Let me read all the key files in parallel.

8. user (2026-06-20T10:03:15.220Z)

```
1      # Rally Annotator
2
3      [![tests](https://github.com/avidullu/rally-
annotator/actions/workflows/ci.yml/badge.svg)](https://github.com/avidullu/rally-
annotator/actions/workflows/ci.yml)
4
5      A tiny VLC plugin to hand-label rally start/end + point-stop reason while you
watch a match,
6      for net-separated racquet sports ? badminton, tennis, table tennis, pickleball,
padel.
```

```

7
8     You watch the video in VLC, pause/scrub freely, and click **Mark START** / **Mark END**
(with a
9     reason). It writes one CSV row per rally ? clean ground-truth labels for
training/evaluating rally
10    (point) segmentation models, or just for cutting highlights.
11
12    ```
13    rally_number,start_time,end_time,ending_reason,sport,shots_count
14    1,8.800,11.500,winner,badminton,9
15    2,24.389,46.589,unforced_error,badminton,21
16    ```
17
18    Times are **decimal seconds**; `shots_count` is an **optional** rally shot/stroke count
(blank if you skip it).
19    See [docs/CSV_FORMAT.md](docs/CSV_FORMAT.md).
20
21    ## Why
22    Labeling rally boundaries is the slow, expensive prerequisite for any rally-detection
model. Most tools
23    make you transcribe timestamps by hand. This lets a rater do it **inside the player they
already use**,
24    pausing to the exact frame before each mark ? turning "watch a match" into "produce
golden labels."
25
26    ## Install
27    1. Copy `vlc/rally_annotator.lua` into your VLC Lua **extensions** folder:
28        - **Windows**: **`%APPDATA%\vlc\lua\extensions\`
29        - **macOS**: **`~/Library/Application Support/org.videolan.vlc/lua/extensions/`
30        - **Linux**: **`~/.local/share/vlc/lua/extensions/`
31        (create the `extensions` folder if it doesn't exist)
32    2. In VLC: **Tools ? Plugins and extensions ? Reload extensions** (or just restart VLC).
33    3. Enable it from the **View** menu ? **Rally Annotator**. A small dialog opens and
stays open while you watch.
34
35    Requires **VLC 3.0.x**. (VLC 4.0 changed the Lua input/listener API; targeting 3.x for
now.)
36
37    ## Use
38    **Playback is built in** ? the **Back 5s · Play / Pause · Fwd 5s** row drives the VLC
player from the
39    annotation window, so you can pause, label, and resume without ever switching to the
main VLC window. **Play / Pause**
40    is a single toggle (it resumes when paused, pauses when playing).
41
42    1. Pick the **Sport** (it stays set across rallies).
43    2. When a rally begins, click **Mark START** ? it snapshots the current playback time
into the **Start** field
44        (pause/scrub first for frame accuracy; you can also edit the field by hand).
45    3. When the rally ends, click **Mark END** (fills the **End** field). This **arms** the
rally; nothing is written yet.
46    4. Choose the **Ending reason** (sits between **Mark END** and **Save Rally**; defaults
to **`unknown`**), optionally
47        type a **Number of shots** (the rally's shot/stroke count ? leave blank to skip),
then click
48        **Save Rally** to write one CSV row. The reason **resets to `unknown` after every
save** (and the shots field
49        clears too) ? so neither silently reuses the previous rally's value, and you can pick
the same reason on
50        consecutive rallies. Use the **Next rally #** field to resume/insert numbering
anywhere.
51    5. **Recent rallies** list: select a row, then **Edit selected** (loads it back into the
fields to fix
52        start/end/reason/sport ? **Save changes**) or **Delete selected**. **Undo last**
removes the most recent row ?

```

53 the button shows which one, e.g. `Undo last (#7)`, and the status panel shows that
row's details ? or clears an
54 in-progress mark, or cancels an edit.

55 6. **Help** button ? usage + an [ending-reason decision guide](docs/ENDING_REASONS.md),
rendered inside the dialog.

56

57 **Output:** `<video-stem>.rallies.csv` next to the video (falls back to your home dir if
the path can't be
58 resolved). Re-opening the extension **continues** rally numbering from the existing file
? no duplicate IDs.

59

60 **Resuming a half-finished video:** labels are saved to that CSV as you go. Open the
same video and enable the
61 extension and it **reloads** your existing rallies (shown in the Recent list) and
continues numbering ? so you can
62 stop and pick up later. Easiest flow: **open** the video first, then enable the
extension. **If** you enable it **before**
63 the video is loaded, that's fine too ? the moment you click **Mark START** (the video is
playing by then) the tool
64 switches to that video's own `<video-stem>.rallies.csv` and loads any rallies already
saved for it, so nothing is
65 written to the home-dir fallback by mistake. If you switch videos with the dialog
already open, click **Refresh** to
66 point at the current video and load its rallies. (Playback *position* isn't restored ?
scrub to where you stopped.)

67

68 > **Where's the help / "About" text?** The empty `"About ?lua"` box under **Tools ?**
Plugins and extensions ?

69 > Add-ons is VLC's own add-on info dialog; its body (`"Lua script"`) is a constant baked
into VLC and **can't** be

70 > set by an extension. Use the **Help** button inside the Rally Annotator dialog
instead. (The descriptor's
71 > description does show in the *Active Extensions* tab's `"More information"` dialog,
and the *Active Extensions*
72 > list shows the title with its version, e.g. `"Rally Annotator v1.6.3"`, so you can
confirm which build is loaded.)

73

74 **## Sports & taxonomy**

75 Net-separated racquet sports share a forced/unforced-error point-stop taxonomy, so one
tool covers them all:

76

Sport	Notes
badminton	the reference sport
tennis	service_fault = fault/double-fault; let supported
table_tennis	let (net serve) supported; fast cadence
pickleball	service_fault for faults; treat "let" per local rules
padel	net-separated; rally boundaries as in tennis

84

85 `ending_reason` ? `{unknown, winner, forced_error, unforced_error, service_fault, let,`
`other}` ? a shared
86 vocabulary across these sports. Keep to these values for clean downstream aggregation.

87

88 **### What the ending reasons mean**

89 `unknown` is the **default** (the field resets to it after every save); every other
reason **except** `winner` is
90 charged to the side that **lost** the rally.

91

Reason	When the rally ended because?
<code>unknown</code>	default ? not classified yet. A save with no reason picked records <code>unknown</code> (never the previous rally's reason); set a specific reason when you can.
<code>winner</code>	the last shot landed in and went unreturned (opponent couldn't reach it / only waved). A clean ace counts here.
<code>forced_error</code>	the loser missed (out or into the net) while under pressure ?

96

```

stretched, rushed, jammed, handling the opponent's pace/spin/depth. |
97     | `unforced_error` | the loser **missed a routine shot** they had time **and** position
to make, with little or no pressure. |
98     | `service_fault` | the point ended **on the serve** ? into the net, out of the service
box, illegal action/foot fault, or a double fault. |
99     | `let` | the rally is **replayed** under the rules, no point scored (e.g. a
tennis/table-tennis serve net-cord that's otherwise good, outside interference). |
100    | `other` | none of the above ? occluded footage, injury/retirement, penalty, hindrance.
Use sparingly. |
101
102    **The cases people ask about:** a ball/shuttle landing **out** is the *hitter's* error ?
`forced_error` only if they
103    were under pressure, otherwise `unforced_error` (it's **never** automatically forced,
and never a winner for the
104    opponent). **Into the net** during a rally is the same; a **serve** into the net is
`service_fault`. When unsure
105    forced-vs-unforced, **default to `unforced_error`**. See the full decision guide with
examples and per-sport rules
106    in **[docs/ENDING_REASONS.md](docs/ENDING_REASONS.md)** (also available via the in-
plugin **Help** button).
107
108    ## Roadmap
109    - [ ] Live-test the v1.6 dialog in VLC (single Play / Pause toggle, optional Number of
shots, two-step Save,
110        `unknown`-default reason, editable times, Next rally #, Recent-rallies
Edit/Delete) across all five sports.
111    - [ ] Optional per-sport reason presets / hotkeys.
112    - [ ] Alternative front-ends for power users / remote raters: a `python-vlc` + Tk/Qt app
with global keyboard
113        shortcuts (S/E/I?G/U), and a zero-install HTML5 `` page that exports the
same CSV.
114    - [x] Optional `shots_count` column (v1.6). Further extra columns (e.g. server/receiver)
still TBD behind a toggle.
115
116    ## Tests
117    The dialog logic has a headless test suite that stubs VLC's `vlc` API (widgets +
playback), loads the real
118    extension, drives its callbacks, and asserts the results ? run with **Lua 5.1** (the
interpreter VLC 3.x embeds):
119
120    ```bash
121    lua5.1 test/dialog_test.lua          # exit 0 = pass; covers reason reset, shots column,
numbering, undo, help toggle, Play/Pause toggle
122    ```
123
124    It covers all the logic but not VLC's GUI rendering or real playback (there's no
headless VLC UI automation) ? those
125    are checked by loading in VLC with `vlc -vv --file-logging` (must scan with no `Error
loading` line) plus a manual
126    click-through. See [test/README.md](test/README.md).
127
128    ## Contributing
129    Issues and PRs welcome ? especially live test reports per sport/OS and small UX fixes.
The plugin is a single
130    self-contained Lua file (`vlc/rally_annotator.lua`) using only documented VLC 3.x Lua
APIs; please keep
131    `test/dialog_test.lua` green (`lua5.1 test/dialog_test.lua`) and add a case for new
dialog logic.
132
133    ## License
134    MIT ? see [LICENSE](LICENSE).
135

```

```

1      # Output CSV format
2
3      One row per rally. Header is written once when the file is created.
4
5      ```
6      rally_number,start_time,end_time,ending_reason,sport,shots_count
7      1,8.800,11.500,winner,badminton,9
8      2,24.389,46.589,unforced_error,badminton,21
9      3,49.183,54.683,let,badminton,
10     ```
11
12     | Column | Type | Notes |
13     |---|---|---|
14     | `rally_number` | int | 1-based, monotonic; continues across re-opening the same CSV |
15     | `start_time` | float | rally start, **decimal seconds** from the video start |
16     | `end_time` | float | rally end, decimal seconds (always `> start_time`) |
17     | `ending_reason` | enum | `unknown` (default) / `winner` / `forced_error` /
`unforced_error` / `service_fault` / `let` / `other` ? see the decision guide in
[ENDING_REASONS.md](ENDING_REASONS.md) |
18     | `sport` | enum | `badminton` / `tennis` / `table_tennis` / `pickleball` / `padel` |
19     | `shots_count` | int | **optional** ? number of shots/strokes in the rally. **Blank**
when not entered (as in row 3 above); added in v1.6. Older CSVs without this column still load.
|
20
21     Notes:
22     - **Decimal seconds**, not `mm:ss` ? downstream loaders parse floats.
23     - Rows are appended live; **Undo last** truncates the last row (one level).
24     - Consumers should read by **column name** (the header is stable but column *order* may
evolve; extra
25     columns may be added later). Most CSV `DictReader`-style loaders already tolerate
extra columns.
26
27     ## Ingesting the labels
28
29     This CSV is intentionally compatible with common rally-segmentation tooling that reads
30     `rally_number,start_time,end_time` by header name and drops rows where `end <= start`:
31
32     ```bash
33     # Generic: any tool that takes a labels CSV of start/end seconds.
34
35     # Example ? score a detector against these labels, or slice clips by them
36     # (badminton-highlight-indexer loaders read --labels <this.csv> directly):
37     python -m backend.eval.calibrate_wasb --trajectory traj.csv --labels match.rallies.csv
--fps 59.94 --frame-width 1920
38     python -m backend.tools.rally_slicer --video match.mp4 --labels match.rallies.csv
--rallies all
39
40     # Example ? register as curated golden data in sports-data-collector:
41     python -m src.cli.curate import-local --sport badminton --video-path match.mp4
--annotations-path match.rallies.csv
42     ```
43
44     The `sport` column is self-documenting; tools that take a `--sport` flag can ignore it.
45

```

10. user (2026-06-20T10:03:16.139Z)

```

1      # Ending reasons ? what they mean & how to label them
2
3      The `ending_reason` column records **why the rally (point) ended**, judged from the
4      **last shot**. There are seven values, shared across all the net-separated racquet
sports
5      this tool supports (badminton, tennis, table tennis, pickleball, padel):
6
7      ```

```

```

8      unknown | winner | forced_error | unforced_error | service_fault | let | other
9      ...
10
11      **`unknown` is the default** ? the annotator resets the reason to `unknown` after every
save, so a
12      rally you didn't classify is honestly recorded as `unknown` (never the previous rally's
reason). Set a
13      specific reason when you can; `unknown` simply marks rallies still needing a verdict.
14
15      **One core principle:** every reason **except `winner`** (and the neutral `unknown`) is
charged to the
16      player/side that **LOST** the rally. `winner` is the only one that credits the side that
**won** it.
17
18      This guide is written for a human rater clicking through a match. Where a rule comes
from
19      the laws of the sport it is marked **[rule]**; where it is an analytics *convention* (a
judgement call that charters agree to apply consistently) it is marked **[convention]**.
20
21
22      ---
23
24      ## The decision procedure (top to bottom ? stop at the first match)
25
26      1. **Was the rally replayed under the rules, with no point scored?**
27      (e.g. a tennis/padel serve that clips the net and lands in the correct box; a table-
tennis
28      serve that touches the net and is otherwise good; outside interference; receiver not
ready.)
29      ? **`let`**. *A net-cord during open rally play that lands in is NOT a let ? keep
going.*
30
31      2. **Did the point end on the SERVE because of a server fault?**
32      (serve into the net, serve outside the correct service court, illegal service action
/ foot
33      fault, or a tennis/padel **double fault**.)
34      ? **`service_fault`**.
35
36      3. **Did the winning side's last shot land IN and go unreturned** ? the opponent
couldn't
37      reach it, or only waved at it without meaningful contact? (A clean untouched serve /
**ace**
38      counts here.) ? **`winner`**.
39
40      4. **Otherwise the rally ended on a MISS by the loser** (ball/shuttle **out**, **into
the net**,
41      or not legally returned). Now judge **pressure**:
42
43      > **The pressure test:** Did the loser have **both enough time AND court position**
to make a
44      > routine return, given the opponent's *immediately preceding* shot (its placement,
pace,
45      > power, spin, depth, angle)? Ask: "Would a typical competent player at this level
be
46      > expected to make that shot?" **[convention]**
47
48      - **Yes** ? had time and position, missed a routine ball ? **`unforced_error`**.
49      - **No** ? time or position was taken away; the miss was induced ?
**`forced_error`**.
50
51      5. **Can't judge it, or it ended off-court** (occluded footage, injury/retirement,
code/point
52      penalty, hindrance, equipment failure) ? **`other`**. Use sparingly.
53
54      **Tie-breaker [convention]:** when you genuinely can't tell forced from unforced,
**default to

```



```

55     `unforced_error`**. Only mark `forced_error` when you can *point to the specific
pressure* (the
56     wide/fast/deep/spinny ball that removed the player's time or position). This keeps
`forced_error`
57     a positively-evidenced label and stops it from inflating. The forced/unforced boundary
is a real
58     *continuum* ? charters disagree on borderline calls, so fix the default in writing for
your team.
59
60     ---
61
62     ## The six reasons, defined
63
64     ### `winner`
65     The point-ending shot landed **in** and the opponent did **not** make a meaningful
return
66     (couldn't reach it, or only waved). Credit to the side that won the rally. A clean
untouched
67     legal serve (an **ace**) is conventionally a `winner`, not a `service_fault`.
68     *Examples:* a smash that bounces untouched; a deceptive drop the opponent never reaches;
a
69     passing shot down the line.
70     *Not a winner:* if the opponent reaches it but mishits **under pressure**, that is
**their**
71     `forced_error`, not your `winner`.
72
73     ### `forced_error`
74     A miss by the **loser** (out, into the net, or not legally returned) where the
opponent's
75     previous shot applied enough pressure ? placement, pace, power, spin, depth, angle ?
that the
76     player did **not** have time *and* position for a routine return. They were *made* to
miss.
77     *Examples:* a wide, heavy serve return netted while lunging at full stretch; a hard
smash dug up
78     but floated long because the player was off-balance and rushed.
79
80     ### `unforced_error`
81     A miss by the **loser** on a shot they had **time AND position** to make routinely, with
little
82     or no pressure. They gave it away.
83     *Examples:* a comfortable mid-court ball drilled into the net; a relaxed clear pushed
past the
84     back line; a sitter smashed long.
85
86     ### `service_fault`
87     A fault by the **server on the serve itself** that loses the point or the serve ? judged
against
88     the sport's *service* rules, not general rally play. Includes: serve into the net, serve
out of
89     the correct service court/box, illegal service action (e.g. badminton contact above 1.15
m,
90     ITTF illegal toss, pickleball illegal motion), foot fault, and a tennis/padel **double
fault**.
91     *Note:* a **first**-serve fault followed by a good second serve is **not** charged here
? only the
92     serve that actually loses the point is. A clean ace is a `winner`, not a
`service_fault`.
93
94     ### `let`
95     A **replay** defined by the laws, where **nobody is charged** a point ? the rally
"didn't count":
96     a tennis/padel service net-cord that still lands in the correct box (replayed, unlimited
times);
97     a table-tennis serve that touches the net and is otherwise correct; an outside

```

```

interruption; the
98     receiver not ready. Use **only** when the rule says *replay*.
99     *Caution:* a net-cord during a **rally** (not the serve) that lands in is live ? **not**
a `let`.
100
101     ### `other`
102     Catch-all for endings that fit none of the above: the outcome is genuinely
unclear/occluded on
103     video; or the point ended administratively (injury retirement, code/point penalty,
hindrance
104     call, equipment failure) rather than on a struck shot. Prefer a specific category
whenever the
105     footage supports it.
106
107     ---
108
109     ## The cases people ask about most
110
111     These are the calls that trip raters up ? resolved precisely.
112
113     ### A. The ball/shuttle lands **OUT** (past the baseline or outside the sidelines)
114     This is **always the hitter's error** ? the side that **lost** the rally. The opponent
merely let
115     it sail (or it cleared the line untouched), so it is **never** the opponent's `winner`.
Decide
116     `forced` vs `unforced` purely by **pressure on the hitter** at the moment they struck it
? look at
117     the opponent's *previous* shot, not the out ball itself:
118     - Rushed, stretched, off-balance, or handling heavy pace/spin/depth ?
**`forced_error`**.
119     - Set and balanced, with time on a routine ball, and still sprayed it ?
**`unforced_error`**.
120
121     "Beyond the baseline" vs "outside the sideline" doesn't change the logic ? both are
*out*, both are
122     the hitter's miss. (In tennis and badminton the **line is in**; this applies only when
it lands
123     fully outside.) **So "out" is not automatically `forced`** ? that label needs visible
pressure;
124     otherwise it's `unforced`.
125
126     ### B. The ball/shuttle goes **INTO the net** (fails to cross)
127     Same as an "out" ball ? it's the hitter's error, and the **pressure test** decides:
`forced_error`
128     if they were rushed/stretched, `unforced_error` if they dumped a routine ball.
129     **Exception:** if the failure-to-cross happens **on the serve**, it is a
**`service_fault`** (in
130     tennis/padel, a missed second serve = double fault), not a rally error ? the rally never
legally
131     began.
132
133     ### C. The **net-cord / net tape** (ball or shuttle clips the top)
134     - **During a rally:** it is **live and good** in all of these sports. Play continues, so
the rally
135     ends on whatever happens **next**: if the opponent can't reach the dribbler ? it's a
`winner`
136     (a "net-cord winner"); if they scramble and then miss ? judge *that* miss as
forced/unforced.
137     A rally net-cord is **never** a `let`.
138     - **On the serve ? sports differ:**
139     | Sport | Serve clips the net and? |
140     |---|---|
141     | **Tennis / Padel** | ?lands in the correct box ? **`let`** (replay). Lands outside ?
`service_fault`. |
142     | **Table tennis** (ITTF) | ?is otherwise correct ? **`let`** (serve replayed). |

```

```

143      | **Badminton** (BWF) | ?passes over and lands in the service court ? **live, play
on** (no service let). But a serve shuttle **caught/suspended on the net** is a
**`service_fault`** (BWF Law 13.2), *not* a let. |
144      | **Pickleball** (2026 USAP) | ?lands in the correct court ? **live, play on** (the
old service-let was removed). Lands out ? `service_fault`. |
145
146      ---
147
148      ## Sport-specific notes
149
150      - **Tennis (ITF):** double fault ? `service_fault`; service net-cord into the box ?
`let`; lines
151      are **in**; a clean ace ? `winner`.
152      - **Badminton (BWF):** service contact must be with the whole shuttle below **1.15 m**
from the
153      court surface (Law 9.1.6; the older "above the waist" wording is the alternative trial
law). A
154      shuttle landing outside the lines or failing to pass the net is a fault; a **serve
caught on the
155      net is a fault** (`service_fault`), while a mid-rally suspended shuttle (Law 14.2.3,
only *after*
156      the serve is returned) is a `let`. Lines are **in**.
157      - **Table tennis (ITTF):** a serve touching the net assembly but otherwise correct is a
**`let`**
158      (replayed); in a rally, a ball that touches the net and still lands on the opponent's
court is a
159      **good return**.
160      - **Pickleball (USAP 2026):** **no** service let ? a net-clip that lands in the correct
court is
161      live; illegal serve motion / net-clip-out ? `service_fault`.
162      - **Padel (FIP):** like tennis for the serve net-cord `let`. Wall play means a rally
ball off the
163      glass after one bounce is still live. (One edge case ? a let-bound serve that touches
the net and
164      then the side **wire mesh** before its 2nd bounce ? is finicky; verify against current
FIP text.)
165
166      ---
167
168      ## Consistency tips
169
170      - The forced/unforced split is a **judgement call on a continuum**. Pick the default
171      (`unforced` when unsure) and apply it the same way every time ? consistency matters
more than
172      any single borderline call.
173      - If you're building a rating *team* or a regression set, consider logging a confidence
note on
174      borderline forced/unforced calls so they can be audited or re-bucketed later.
175      - These six values are deliberately a small, shared vocabulary. Keep to them (don't
invent new
176      spellings) so downstream aggregation stays clean.
177
178      ### Mapping to tennis charting codes (optional)
179      If you cross-reference the [Match Charting
Project](https://github.com/JeffSackmann/tennis_MatchChartingProject):
180      `winner` = *, `forced_error` = #, `unforced_error` = @ (with location tags `n` = net,
`w` = wide,
181      `d` = deep, `x` = wide+deep). These are charting conventions layered on top of the
official rules.
182
183      ---
184
185      ## Sources
186
187      - Match Charting Project ? quick-start & codes:

```

<<https://www.tennisabstract.com/blog/2015/09/23/the-match-charting-project-quick-start-guide/>>
188 - "The continuum of errors" (forced vs unforced is a continuum):
<<http://www.tennisabstract.com/blog/2017/01/13/the-continuum-of-errors/>>
189 - Leo Levin's pressure definition of unforced error:
<<https://www.tennis.com/news/articles/what-is-an-unforced-error-a-meditation-on-the-tennis-player-s-least-favorite-stat>>
190 - ITF Rules of Tennis: <<https://www.itftennis.com/en/about-us/governance/rules-and-regulations/>>
191 - BWF Laws of Badminton: <<https://www.worldbadminton.com/rules/>>
192 - ITTF Table Tennis rules (Law 2.7 / 2.9): <<https://www.ittf.com/handbook/>>
193 - USA Pickleball rulebook: <<https://usapickleball.org/what-is-pickleball/official-rules/>>
194 - FIP Rules of Padel: <<https://www.padelfip.com/rules-of-padel/>>
195

11. user (2026-06-20T10:03:16.616Z)

```
1 MIT License
2
3 Copyright (c) 2026 Avi Dullu
4
5 Permission is hereby granted, free of charge, to any person obtaining a copy
6 of this software and associated documentation files (the "Software"), to deal
7 in the Software without restriction, including without limitation the rights
8 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
9 copies of the Software, and to permit persons to whom the Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be included in all
13 copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
21 SOFTWARE.
22
```

12. user (2026-06-20T10:03:16.932Z)

```
1 # Changelog
2
3 ## v1.6.4 ? 2026-06-18
4 - **Guard unsaved work against silent loss (golden-data safety).** Once a rally is fully
5 marked (**START + END**)
6 but not yet saved, the two ways you could *accidentally* throw it away are now blocked
7 with a clear prompt instead
8 of silently discarding it:
9 - **Mark START** (starting a new rally) is refused ? "Save Rally to keep it, or Undo
10 last to clear it" ? so a new
11 mark can't overwrite an unsaved one.
12 - **Edit selected** is refused for the same reason (loading another row would replace
13 the unsaved START?END).
14 (The Start/End fields stay editable by hand, and **Undo last** is the explicit one-
15 click "clear the mark" escape.)
16 - **Edit mode is now unmissable.** While editing an existing rally, the Mark buttons
17 relabel to
18 ***"Re-mark START (#N)" / "Re-mark END (#N)"**, so it's obvious they change *that*
19 rally rather than creating a new
20 one (they revert to "Mark START"/"Mark END" when the edit is saved or cancelled).
21 - Regression tests for both guards (refusal + the relabel/revert) ? suite now **65
22 assertions**.
23
```

```

16      ## v1.6.3 ? 2026-06-18
17      - **The version now shows next to the plugin in VLC's "Active Extensions" list** (Tools
? Plugins and extensions ?
18      Active Extensions) ? e.g. **`Rally Annotator v1.6.3`** ? matching how VLsub shows
`VLsub 0.11.1`. VLC renders the
19      descriptor `title` verbatim there and never appends the version, so the version is now
baked into the title, fed by
20      the same `VERSION` constant as the dialog window title and `descriptor().version` (so
all three stay in sync).
21      Regression test asserts `descriptor().title == "Rally Annotator v" .. version`.
22
23      ## v1.6.2 ? 2026-06-18
24      - **Fix (data safety on resume): rallies could be written to `~/rally_labels.csv`
instead of next to the video.**
25      `out_path` was resolved **once**, when the extension was enabled. If the video wasn't
loaded at that instant
26      (extension enabled before the video, or VLC still opening it), `current_media_path()`
returned `nil` and the path
27      fell back to the home dir ? so labels were saved there, and on **pause/restart** the
video's own
28      `

```

```

(`test/dialog_layout.snapshot`) is
54     regenerated for the new grid, and the suite grows to **45 assertions** (shots
write/blank/edit, Play/Pause toggle),
55     green via `lua5.1 test/dialog_test.lua`.
56
57     ## v1.5.1 ? 2026-06-09
58     - **UX: the annotation window title now shows the version** ? the dialog opens as
***"Rally Annotator v1.5.1"*** instead
59     of just "Rally Annotator", so a rater can tell at a glance which build they're
running. The version is a single
60     `VERSION` constant the descriptor and the dialog title both read, so they can't drift.
61     - **Test:** added two assertions that the dialog title carries the version and exactly
matches `Rally Annotator v` +
62     `descriptor().version`.
63     - **Test: layout snapshot** (`test/dialog_layout.snapshot`). The harness now serializes
the full widget grid the
64     extension hands to VLC ? every control's kind, grid column/row, column/row span, and
caption/options, plus the
65     window title ? and diffs it against a committed golden. It's the deterministic, cross-
platform stand-in for a
66     screenshot diff (VLC renders extension dialogs through Qt with no headless path),
catching any moved, resized,
67     renamed, added, or removed control and any title change. Regenerate intentional layout
changes with
68     `lua5.1 test/dialog_test.lua --update`. **39 assertions total.**
69     - **CI: GitHub Actions** (.github/workflows/ci.yml) runs on every push and PR ?
syntax-checks the extension with
70     `luac5.1 -p` (the kind of scan-time error that silently broke loading in v1.3.1), then
runs the full dialog suite
71     including the layout snapshot. A status badge is on the root `README`.
72     - **Docs:** `test/README.md` gains a **Windows** Lua-5.1 install FAQ (no-admin portable
install + the `choco` route),
73     since `lua5.1` isn't on PATH by default there.
74
75     ## v1.5 ? 2026-06-08
76     - **Feature: playback controls (issue #4).** A new **Back 5s · Play / Resume · Pause ·
Fwd 5s** row drives the VLC
77     player from the annotation window ? pause, label, and resume without switching to the
main VLC window. Uses the
78     extension-exposed `vlc.playlist` (play/pause/status) and `vlc.var` seek on the input;
Pause / Play-Resume are
79     **gated on `vlc.playlist.status()`** because VLC's `pause()` is a hard toggle, so they
never flip the wrong way.
80     Seek is relative  $\pm 5$  s, clamped at 0. (Verified feasible against the VLC 3.0 source
before building.)
81     - **Committed test suite** (`test/dialog_test.lua`, run with `lua5.1`): stubs VLC's
dialog + playback API, loads the
82     real extension, and asserts reason reset / consecutive-reason / numbering / undo re-
sync / help toggle / playback
83     gating / CSV output ? **36 assertions, exit-code-based** for CI. See `test/README.md`.
84     - Dialog regrouped to fit the playback row at the top; reason dropdown cell `(3,4)` ?
`(3,5)`, help panel row `14` ? `15`.
85
86     ## v1.4 ? 2026-06-08
87     - **Fix: the Help button now toggles reliably.** It previously got "stuck" showing help
? VLC didn't repaint the
88     in-place text swap on the shared status widget. Help is now a **dedicated panel**
added/removed via
89     `add_html`/`del_widget`, which forces a real layout change VLC renders every time.
(Verified with a headless
90     `lua5.1` harness that stubs the VLC dialog API and drives the toggle.)
91     - **Feature: "Next rally #" field** ? choose where numbering resumes from (restart at 1,
continue from 50, fill a
92     gap, ?). Defaults to the natural next number, auto-advances to the next **free**
number after each save, and

```

```

93         refuses a number that already exists in the CSV. The Save button shows the number it
will write.
94     - **Fix: the Ending reason no longer gets "stuck".** Picking a reason (e.g. `winner`)
used to block the next rally
95         from using the same one ? VLC didn't repaint the `clear()`-based dropdown reset. The
reason now resets to a real,
96         savable **`unknown`** default by recreating the dropdown (same `del_widget` technique
as Help), so consecutive
97         rallies can share a reason and a forgotten pick records `unknown` rather than the
previous rally's reason. Reason
98         is no longer a hard-required field; `unknown` was added to the documented vocabulary.
99     - **Fix: removing a rally no longer leaves a numbering gap.** `Undo last` and `Delete
selected` now re-sync the
100         "Next rally #" field to the current data.
101
102     ## v1.3.1 ? 2026-06-08
103     - **Fix: the extension failed to load and never appeared under the View menu.** v1.3
built the `REASON_ID`/`SPORT_ID`
104         lookup tables with **top-level `ipairs` loops**, which run during VLC's *descriptor
scan* ? a restricted Lua sandbox
105         that does not expose base globals like `ipairs` ? so the whole script errored at scan
time
106         (`attempt to call global 'ipairs' (a nil value)`) and VLC skipped it. Moved that
table-building into `activate()`
107         (the full Lua environment); no top-level code now calls any global function. Verified
via VLC's `-vv` extension-scan log.
108
109     ## v1.3 ? 2026-06-08
110     - **Ending reason is now required and non-sticky:** a `-- choose reason --` placeholder
is the default, Save is
111         refused until a real reason is picked, and the reason **resets after every save**
(fixes the bug where the
112         previous rally's reason was silently reused).
113     - **Two-step commit:** `Mark END` now *arms* the rally; you pick the reason and click
**Save Rally** to write it
114         (the button shows the rally number it will write). Per-rally reasons are captured at
save time.
115     - **Editable Start/End fields** ? fine-tune the marked seconds before saving.
116     - **Edit recent rallies:** a *Recent rallies* list with **Edit selected** / **Delete
selected**; `Undo last`
117         removes the most recent row, clears an in-progress mark, or cancels an edit. `Undo
last` now shows **which** row
118         it will remove (`Undo last` (#N)`) and the status panel shows that row's details. CSV
is now rewritten atomically
119         (tmp + rename, with a Windows `.bak` rollback) and preserves any extra columns.
120     - **Resume a half-finished video:** on activate the extension reloads that video's
existing `.rallies.csv` (shown
121         in the Recent list, with continued numbering); **Refresh** now re-points to whatever
video is playing and loads
122         its rallies ? so reopening (or switching) videos picks up where you left off.
(Playback position isn't restored.)
123     - **In-dialog Help button** ? usage + an ending-reason decision guide, rendered inside
the extension's own dialog
124         (VLC's own *About* dialog shows a hardcoded "Lua script" and cannot be set by an
extension). New repo guide
125         `.docs/ENDING_REASONS.md`; README gains reason definitions and the out-of-bounds /
into-net / net-cord rulings.
126     - Richer `descriptor()` `shortdesc`/`description` (these surface in the *Active
Extensions* tab's "More information").
127
128     ## v1.2 ? 2026-06-07
129     - Generalized from badminton-only to **net-separated racquet sports** (badminton,
tennis, table tennis,
130         pickleball, padel) via a new **Sport** dropdown and a `sport` CSV column.
131     - Generic naming/branding; MIT-licensed standalone release.

```

```

132
133     ## v1.1 (badminton-only, pre-release; from badminton-highlight-indexer)
134     - Button-driven dialog (not an auto-pause hook ? that VLC callback is flaky on macOS /
broken in VLC 4.0).
135     - Snapshots `vlc.var.get(input,"time")/1e6` (microseconds ? seconds).
136     - Continues rally numbering across re-enable (no duplicate `rally_number`); one-level
Undo; HTML status panel.
137     - Writes `rally_number,start_time,end_time,ending_reason` next to the video.
138
139     > Note: v1.2 changes the multi-sport path and is **pending a live VLC smoke test**
across all five sports.
140

```

13. assistant (2026-06-20T10:03:19.664Z)

Now let me read the source code (the Lua extension) to verify the actual implementation, plus check for any screenshots/images and the test files.

14. user (2026-06-20T10:03:20.636Z)

```

1      --[[ rally_annotator.lua  --  VLC Lua EXTENSION (v1.6.4)
2
3      Rally Annotator for NET-SEPARATED RACQUET SPORTS
4      (badminton · tennis · table tennis · pickleball · padel)
5
6      While watching a match in VLC, mark each rally's START / END and the point-stop
7      reason, and append one CSV row per rally. Pause/scrub freely, then click ? the
8      callback snapshots the exact playback time. Output is a plain CSV that ingests
9      directly into common rally-segmentation tooling.
10
11      Output CSV columns (times in decimal SECONDS):
12          rally_number,start_time,end_time,ending_reason,sport,shots_count
13
14      WHAT'S NEW IN v1.6.4
15      - Guard against silently losing unsaved work: once a rally is fully marked
16        (START + END) but not yet saved, clicking Mark START (new rally) or Edit
17        selected is REFUSED with a prompt to Save Rally or Undo last first.
18      - While editing an existing rally, the Mark buttons relabel to
19        "Re-mark START/END (#N)" so it's obvious they change THAT rally, not a new one.
20
21      WHAT'S NEW IN v1.6.3
22      - The version now shows next to the plugin name in VLC's "Active Extensions"
23        list (Tools > Plugins and extensions), e.g. "Rally Annotator v1.6.3" --
24        VLC shows the descriptor title verbatim, so the version is baked into it.
25
26      WHAT'S NEW IN v1.6.2
27      - Fix (data safety on resume): if the extension was enabled BEFORE the video
28        was loaded, the CSV path fell back to ~/rally_labels.csv and rallies were
29        written there instead of next to the video -- so on pause/restart the
30        video's labels looked "lost". Now the moment you Mark START (video is
31        provably playing), the tool adopts <video>.rallies.csv and loads any
32        rallies already saved for it.
33
34      WHAT'S NEW IN v1.6.1
35      - Fix: the Recent rallies list now shows EVERY rally (it scrolls), not just
36        the most recent 12 -- so the oldest rallies (e.g. #1) are no longer hidden
37        and can still be selected for Edit/Delete.
38
39      WHAT'S NEW IN v1.6
40      - Playback is now a single PLAY / PAUSE toggle (was separate Play/Resume +
41        Pause), so the playback row is just Back 5s / Play / Pause / Fwd 5s.
42      - New optional "Number of shots" field -> a shots_count column appended to
43        the CSV (blank when you don't fill it; older CSVs still load fine).
44
45      WHAT'S NEW IN v1.3

```



```

46     - In-dialog HELP button: usage + an ending-reason decision guide, rendered
47       inside the extension's own dialog (see "WHERE HELP LIVES" below).
48     - Two-step commit: Mark END now ARMS a rally; you pick the (required) reason
49       and click "Save Rally" to write it. The reason is RESET after every save,
50       so it can never silently reuse the previous rally's reason.
51     - Editable Start/End fields (fine-tune the marked seconds before saving).
52     - "Recent rallies" list with Edit selected / Delete selected, plus Undo last.
53
54 WHERE HELP LIVES (VLC internals, verified against the 3.0.x source):
55     The empty "About <name>.lua" window under Tools > Plugins and extensions >
56     Add-ons is VLC's AddonInfoDialog; its body text ("Lua script") is a hardcoded
57     constant supplied by VLC's filesystem addon scanner (modules/misc/addons/
58     fsstorage.c) and CANNOT be set by an extension. The descriptor() shortdesc/
59     description below DO show, but in a different place ? the "More information"
60     dialog reached from the *Active Extensions* tab. For real, author-controlled
61     help we therefore render it INSIDE this extension's dialog (the HELP button).
62
63 WHY a button-dialog (not an auto-pause hook):
64     VLC 3.x can expose pause via the {"playing-listener"} capability
65     (status_changed/playing_changed), but that callback is flaky on macOS
66     (VLC #22778) and the input/meta listeners are broken in VLC 4.0 (#27558).
67     A persistent dialog whose button callbacks SNAPSHOT the current playback
68     time is portable, precise, and lets the rater pause/scrub before committing.
69     So we declare NO listener capabilities.
70
71 UNITS: in VLC 3.x, vlc.var.get(input,"time") is in MICROSECONDS -> /1e6.
72
73 Widget grid args are (col, row, colspan, rowspan), 1-indexed.
74
75 Install (Windows): copy this file to
76     %APPDATA%\vlc\lua\extensions\rally_annotator.lua
77     macOS: ~/Library/Application Support/org.videolan.vlc/lua/extensions/
78     Linux: ~/.local/share/vlc/lua/extensions/
79     then VLC > Tools > Plugins and extensions > Reload extensions (or restart),
80     then enable it from the View menu.
81
82 MIT licensed. https://github.com/avidullu/rally-annotator
83 ]]
84
85 -----
86 -- Extension registration
87 -----
88 local VERSION = "1.6.4"
89
90 function descriptor()
91     return {
92         -- VLC's "Active Extensions" list shows the title VERBATIM (it never appends the
93         -- version), so we bake the version into the title -- same as VLSub -- to show it
94         -- next to the plugin name there. Fed by VERSION so it can't drift. (Concatenation
95         -- only: descriptor() runs in VLC's restricted scan sandbox, which has no globals.)
96         title      = "Rally Annotator v" .. VERSION,
97         version    = VERSION,
98         author     = "Avi Dullu",
99         url        = "https://github.com/avidullu/rally-annotator",
100        shortdesc   = "Mark rally start/end + a point-ending reason to a CSV (net-separated
101        racquet sports)",
102        description =
103            "Mark each rally's START and END while you watch, tag WHY the point ended "
104            .. "(unknown / winner / forced_error / unforced_error / service_fault / let /
105            other), and append "
106            .. "one CSV row per rally next to the video "
107            .. "(rally_number,start_time,end_time,ending_reason,sport,shots_count; decimal
108            seconds). "
109            .. "Built-in Play/Pause + seek so you never leave the window; "
110            .. "two-step Save (Mark END, then Save Rally) with a non-sticky reason (defaults to

```

```

unknown); "
108     .. "editable times; edit or delete recent rallies; resumable numbering. "
109     .. "For badminton/tennis/table-tennis/pickleball/padel. "
110     .. "Click the HELP button inside the dialog for usage + an ending-reason decision
guide.",
111     capabilities = {}    -- button-click model: no listeners needed
112 }
113 end
114
115 -----
116 -- Config / constants
117 -----
118 -- Net-separated racquet sports share a forced/unforced-error point-stop taxonomy.
119 -- "unknown" is the default/reset value: a real, savable reason meaning "not yet
120 -- classified" (so a forgotten pick records 'unknown', never the previous rally's
reason).
121 local REASON_DEFAULT = "unknown"
122 local REASONS = {
123     "winner", "forced_error", "unforced_error", "service_fault", "let", "other"
124 }
125 local SPORTS = {
126     "badminton", "tennis", "table_tennis", "pickleball", "padel"
127 }
128 local HEADER = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
129
130 -- reason/sport -> id lookups. POPULATED IN activate(), NOT here: VLC scans an
131 -- extension's descriptor() in a restricted Lua sandbox that lacks base globals
132 -- like ipairs, so calling ipairs at top level makes the whole extension fail to
133 -- load (it then never appears under the View menu).
134 local REASON_ID = {}
135 local SPORT_ID = {}
136
137 -- In-dialog help (rendered into the status panel by the HELP button). Basic HTML
138 -- only (<b>, <i>, <br>) for portability across the Qt and macOS dialog renderers.
139 local HELP_HTML = [=[
140 <b>Rally Annotator &mdash; how to use</b><br>
141 <b>Playback:</b> the <b>Back 5s / Play / Pause / Fwd 5s</b> row drives the VLC player
from here &mdash; <b>Play / Pause</b> is one toggle (pause, annotate, and resume without
switching to the main VLC window).<br>
142 1. Pick the <b>Sport</b> (top). It stays set across rallies.<br>
143 2. When a rally begins, click <b>Mark START</b> (pause/scrub first for frame accuracy
&mdash; it snapshots the current playback time into the Start field).<br>
144 3. When the rally ends, click <b>Mark END</b>. You may fine-tune the Start/End seconds
by editing those fields directly.<br>
145 4. Choose the <b>Ending reason</b> (the box between <b>Mark END</b> and <b>Save
Rally</b>; or leave it as <b>unknown</b>), optionally type a <b>Number of shots</b> (the rally's
shot/stroke count &mdash; leave blank to skip), then click <b>Save Rally</b> &mdash; one CSV row
is written next to the video.<br>
146 5. The reason RESETS to <b>unknown</b> after every save (never silently reused), and you
can pick the same reason on consecutive rallies.<br>
147 6. <b>Recent rallies</b>: select a row, then <b>Edit selected</b> (loads it back into
the fields) or <b>Delete selected</b>. <b>Undo last</b> removes the most recent row (the button
shows which, e.g. <i>Undo last (#7)</i>), or clears an in-progress mark, or cancels an edit.<br>
148 7. <b>Resuming later:</b> labels are saved to the CSV next to the video as you go. Re-
open the SAME video and enable the extension &mdash; it reloads your existing rallies and
continues numbering. Set the <b>Next rally #</b> field to resume numbering from any value (e.g.
restart at 1, or continue from 50); it auto-advances after each save. If you switch videos with
this dialog open, click <b>Refresh</b> to load the current video's rallies. (The video's
playback position is not restored &mdash; scrub to where you stopped.)<br>
149 <br>
150 <b>Ending reasons &mdash; what they mean (pick the one that says WHY the rally
ended).</b><br>
151 All reasons except <i>winner</i> are charged to the side that LOST the rally.<br>
152 &bull; <b>winner</b> &mdash; the last shot landed IN and was not returned (opponent
couldn't reach it, or only waved at it). A clean untouched serve (ace) counts here.<br>

```

```

153      &bull; <b>forced_error</b> &mdash; the loser MISSED (out, or into the net) while UNDER
PRESSURE: stretched, rushed, jammed, or handling the opponent's pace/spin/depth. They were made
to miss.<br>
154      &bull; <b>unforced_error</b> &mdash; the loser MISSED a routine shot they had time AND
position to make, with little or no pressure. They gave it away.<br>
155      &bull; <b>service_fault</b> &mdash; the point ended on the SERVE: serve into the net,
out of the service box, illegal action/foot fault, or a double fault (tennis/padel).<br>
156      &bull; <b>let</b> &mdash; the rally is REPLAYED under the rules (no point): e.g. a
tennis/table-tennis serve that clips the net and is otherwise good, or outside interference.<br>
157      &bull; <b>other</b> &mdash; anything else (occluded footage, injury/retirement, penalty,
hindrance). Use sparingly.<br>
158      &bull; <b>unknown</b> &mdash; the default; this rally hasn't been classified yet (pick a
specific reason when you can).<br>
159      <br>
160      <b>Quick cases</b><br>
161      &bull; Ball/shuttle lands <b>OUT</b> (past the baseline / outside the lines): it is the
hitter's miss &mdash; <b>forced</b> if they were under pressure, <b>unforced</b> if it was a
comfortable ball. (OUT is NOT automatically forced, and never a winner for the opponent.)<br>
162      &bull; <b>Into the net</b> during a rally: same &mdash; forced if pressured, unforced if
routine. A <b>serve</b> into the net is <b>service_fault</b>.<br>
163      &bull; <b>Net-cord that dribbles over and lands in</b>: live and good &mdash; usually a
<b>winner</b> if unreachable, else judge what happens next. On the SERVE it varies by sport:
tennis/table-tennis = <b>let</b> (replay); badminton net-tick that passes over and lands in =
play on (but a serve shuttle CAUGHT on the net = service_fault); pickleball (2026) = play
on.<br>
164      &bull; <b>Unsure forced vs unforced?</b> Default to <b>unforced</b> &mdash; only mark
forced when you can point to the specific pressure.<br>
165      <br>
166      Output: <i>&lt;video>.rallies.csv</i> next to the video. Full guide:
docs/ENDING_REASONS.md in the repo.<br>
167      Click <b>Hide help</b> to return to the status panel.
168      ]==]
169
170      -----
171      -- State
172      -----
173      local d                -- the single dialog (one per extension)
174      local w_status         -- status label widget (HTML/rich-text); also hosts help
175      local w_reason         -- ending_reason dropdown widget
176      local w_sport          -- sport dropdown widget
177      local w_start          -- start-time text input (editable seconds)
178      local w_end            -- end-time text input (editable seconds)
179      local w_list           -- recent-rallies list widget
180      local w_save           -- "Save Rally" / "Save changes" button (reabeled by state)
181      local w_mark_start     -- "Mark START" button (reabeled "Re-mark START (#N)" while
editing)
182      local w_mark_end       -- "Mark END" button (reabeled "Re-mark END (#N)" while
editing)
183      local w_undo           -- "Undo last" button (reabeled to show which row it removes)
184      local w_next           -- "Next rally #" text input (lets you resume numbering
anywhere)
185      local w_shots          -- "Number of shots" text input (optional shots_count per rally)
186      local w_help_btn       -- the Help button (reabeled "Hide help" when open)
187      local w_help           -- the help panel widget; nil when hidden (toggled via
del_widget)
188
189      local rows = {}        -- in-memory rally rows: { n, s, e, reason, sport, [shots],
[extra] }
190      local mode = "new"     -- "new" (marking a fresh rally) or "edit" (editing a row)
191      local edit_index = nil -- index into rows when mode == "edit"
192      local out_path         -- resolved CSV path (set on activate; re-resolved by Refresh)
193      local out_path_is_fallback = false -- true when out_path is the home-dir fallback (no
video
194                                     -- was resolvable when we last set it); see
adopt_current_video_csv

```

```

195
196 -----
197 -- Helpers (declared before the global callbacks that capture them)
198 -----
199
200 -- Current playback time in SECONDS, or nil if nothing is playing.
201 local function now_seconds()
202     local input = vlc.object.input()
203     if not input then return nil end
204     local t_us = vlc.var.get(input, "time") -- MICROSECONDS in VLC 3.x
205     if not t_us then return nil end
206     return t_us / 1000000.0
207 end
208
209 -- mm:ss.mmm for display only.
210 local function fmt_clock(s)
211     if not s then return "--:--" end
212     if s < 0 then s = 0 end
213     local m = math.floor(s / 60)
214     local sec = s - m * 60
215     return string.format("%d:%06.3f", m, sec)
216 end
217
218 -- Minimal HTML-escape so paths/messages render literally in the rich-text label.
219 local function esc(text)
220     text = tostring(text or "")
221     text = text:gsub("&", "&amp;")
222     text = text:gsub("<", "&lt;")
223     text = text:gsub(">", "&gt;")
224     return text
225 end
226
227 -- Best-effort path to the currently playing media file (decoded from URI).
228 local function current_media_path()
229     local input = vlc.object.input()
230     if not input then return nil end
231     local item = vlc.input.item() -- VLC 3.x: vlc.input.item()
232     if not item then return nil end
233     local uri = item.uri() -- item.uri()
234     if not uri or uri == "" then return nil end
235     local p = uri
236     -- Strip the scheme. Windows file URIs look like file:///C:/dir/clip.mp4;
237     -- POSIX ones like file:///home/user/clip.mp4 -- keep the POSIX leading "/".
238     if p:match("^file:///a:") then
239         p = p:gsub("^file:///", "") -- file:///C:/... -> C:/...
240     else
241         p = p:gsub("^file://", "") -- file:///home/... -> /home/...
242     end
243     -- percent-decode (e.g. %20 -> space) BEFORE separator normalization
244     p = p:gsub("%(%x%x)", function(h) return string.char(tonumber(h, 16)) end)
245     if p:match("^a:") then -- Windows drive path -> backslashes
246         p = p:gsub("/", "\\")
247     end
248     return p
249 end
250
251 -- Resolve a sensible default output path: next to the video if we can, else
252 -- the user's home (USERPROFILE on Windows, HOME elsewhere).
253 local function resolve_out_path()
254     local media = current_media_path()
255     if media then
256         local dir, stem = media:match("^([^\\\/])([\\\/-]?%.[^\\\/%.]*$")
257         if dir and stem and stem ~= "" then
258             return dir .. stem .. ".rallies.csv"
259         end

```

```

260     local d2 = media:match("^([\\/]")
261     if d2 then return d2 .. "rally_labels.csv" end
262 end
263 local home = os.getenv("USERPROFILE") or os.getenv("HOME") or "."
264 local sep = home:find("\\") and "\\" or "/"
265 if home:sub(-1) ~= sep then home = home .. sep end
266 return home .. "rally_labels.csv"
267 end
268
269 -- Load all rally rows from the CSV into `rows` (header + blank lines skipped).
270 -- Forgiving parser: any leading-integer line is a rally row. Column 6 is the
271 -- optional shots_count (blank/missing in older CSVs); any columns beyond it are
272 -- preserved verbatim so we never drop richer metadata on rewrite.
273 local function load_rows()
274     rows = {}
275     local f = io.open(out_path, "r")
276     if not f then return end
277     for line in f:lines() do
278         line = line:gsub("\r$", "")
279         if line:match("%S") then
280             local parts = {}
281             for field in (line .. ","):gmatch("([^\,]*)") do parts[#parts + 1] = field end
282             local n = tonumber(parts[1])
283             if n and n == math.floor(n) then
284                 local shots = parts[6]
285                 if shots == "" then shots = nil end -- blank => not recorded
286                 local row = {
287                     n = math.floor(n),
288                     s = tonumber(parts[2]) or 0,
289                     e = tonumber(parts[3]) or 0,
290                     reason = (parts[4] ~= nil and parts[4] ~= "") and parts[4] or "other",
291                     sport = parts[5] or "",
292                     shots = shots,
293                 }
294                 if #parts > 6 then
295                     local ex = {}
296                     for i = 7, #parts do ex[#ex + 1] = parts[i] end
297                     row.extra = table.concat(ex, ",")
298                 end
299                 rows[#rows + 1] = row
300             end
301         end
302     end
303     f:close()
304 end
305
306 -- Rewrite the whole CSV from `rows` (header once). We write a temp file and swap
307 -- it into place so the live CSV is never lost if a step fails: rename is atomic on
308 -- POSIX (and succeeds for a brand-new file on Windows); when Windows can't rename
309 -- over an existing file we stash the original as .bak and roll back on error.
310 local function save_all()
311     local tmp = out_path .. ".tmp"
312     local f, err = io.open(tmp, "w")
313     if not f then return false, ("cannot open CSV for write: " .. tostring(err)) end
314     f:write(HEADER)
315     for _, r in ipairs(rows) do
316         local shots = (r.shots ~= nil) and tostring(r.shots) or ""
317         local line = string.format("%d,%.3f,%.3f,%s,%s,%s", r.n, r.s, r.e, r.reason,
r.sport, shots)
318         if r.extra and r.extra ~= "" then line = line .. "," .. r.extra end
319         f:write(line .. "\n")
320     end
321     f:close()
322     -- Atomic on POSIX; also succeeds on Windows when out_path doesn't exist yet.
323     if os.rename(tmp, out_path) then return true end

```

```

324     -- Windows overwrite path: keep a backup so the original is never lost.
325     local bak = out_path .. ".bak"
326     os.remove(bak)
327     local stashed = os.rename(out_path, bak)          -- move original aside (nil if none)
328     local ok, rerr = os.rename(tmp, out_path)
329     if not ok then
330         if stashed then os.rename(bak, out_path) end -- roll back
331         return false, ("cannot replace CSV: " .. tostring(rerr))
332     end
333     os.remove(bak)
334     return true
335 end
336
337 -- Next monotonic rally_number (max existing + 1); keeps numbering across reopen.
338 local function next_rally_number()
339     local maxn = 0
340     for _, r in ipairs(rows) do if r.n > maxn then maxn = r.n end end
341     return maxn + 1
342 end
343
344 -- Read a text-input widget as a number, or nil if blank/non-numeric.
345 local function get_field_num(w)
346     if not w then return nil end
347     local t = w:get_text()
348     if not t then return nil end
349     t = t:gsub("%s+", "")
350     if t == "" then return nil end
351     return tonumber(t)
352 end
353
354 -- The "Number of shots" field as a non-negative integer, or nil if blank or not a
355 -- valid count. Optional metadata: a blank/garbage value just records no shots_count.
356 local function get_shots()
357     local v = get_field_num(w_shots)
358     if not v or v < 0 then return nil end
359     return math.floor(v)
360 end
361
362 -- True when a fresh rally is fully marked (both START and END) but NOT yet saved --
363 -- i.e. there is unsaved work in the fields. Guards against silently discarding it when
364 -- the user starts a new rally (Mark START) or jumps to editing another (Edit selected).
365 local function is_armed()
366     return mode == "new" and get_field_num(w_start) ~= nil and get_field_num(w_end) ~= nil
367 end
368
369 -- The number the NEXT new rally will get. Defaults to next_rally_number(), but the
370 -- user can type any value into the "Next rally #" field to resume/insert anywhere.
371 local function planned_next_number()
372     local v = get_field_num(w_next)
373     if v and v >= 1 then return math.floor(v) end
374     return next_rally_number()
375 end
376
377 -- Reset the "Next rally #" field to the natural next number for the current CSV.
378 local function refresh_next_field()
379     if w_next then w_next:set_text(tostring(next_rally_number())) end
380 end
381
382 local function index_of_rally(n)
383     for i, r in ipairs(rows) do if r.n == n then return i end end
384     return nil
385 end
386
387 -- Smallest integer >= start that is not already a rally_number (auto-advance skips
388 -- over occupied numbers so consecutive saves never walk into an existing one).

```

```

389     local function next_free_from(start)
390         local n = start
391         while index_of_rally(n) do n = n + 1 end
392         return n
393     end
394
395     -- First selected rally_number in the recent list, or nil. get_selection() returns
396     -- a { [id]=text } table in VLC 3.0.x (id is the rally_number we stored).
397     local function selected_rally_number()
398         if not w_list then return nil end
399         local sel = w_list:get_selection()
400         if type(sel) ~= "table" then return nil end
401         for id, _ in pairs(sel) do
402             return tonumber(id) or id
403         end
404         return nil
405     end
406
407     -- Dropdowns have no set_value in 3.0.x. clear()+re-add did NOT repaint reliably in
408     -- VLC (the selection got "stuck" on the previous reason), so we RECREATE the dropdown
409     -- at the SAME grid cell via del_widget+add_dropdown -- a real widget swap VLC renders
410     -- every time, with no net layout change. The first value added is auto-selected, so we
411     -- add the value we want shown first.
412     local function rebuild_reason_default()
413         if not d then return end
414         if w_reason then d:del_widget(w_reason) end
415         w_reason = d:add_dropdown(3, 6, 1, 1)
416         w_reason:add_value(REASON_DEFAULT, 0) -- "unknown" default (id 0)
417         for i, v in ipairs(REASONS) do w_reason:add_value(v, i) end
418     end
419
420     local function rebuild_reason_selected(sel)
421         if not d then return end
422         if not sel or sel == "" then sel = REASON_DEFAULT end
423         if w_reason then d:del_widget(w_reason) end
424         w_reason = d:add_dropdown(3, 6, 1, 1)
425         w_reason:add_value(sel, REASON_ID[sel] or 0) -- selected first
426         if sel ~= REASON_DEFAULT then w_reason:add_value(REASON_DEFAULT, 0) end
427         for i, v in ipairs(REASONS) do
428             if v ~= sel then w_reason:add_value(v, i) end
429         end
430     end
431
432     local function rebuild_sport_selected(sel)
433         if not w_sport then return end
434         if not sel or sel == "" then sel = SPORTS[1] end
435         w_sport:clear()
436         local id = SPORT_ID[sel] or 99
437         w_sport:add_value(sel, id) -- first => shown selected
438         for i, v in ipairs(SPORTS) do
439             if v ~= sel then w_sport:add_value(v, i) end
440         end
441     end
442
443     -- Repaint the recent-rallies list with EVERY rally (oldest first). The box is a
444     -- fixed-height, scrollable list, so we show the full history instead of a window --
445     -- a "last 12" cap used to hide the oldest rallies (e.g. #1 once you had 13+), which
446     -- also made them impossible to select for Edit/Delete.
447     local function refresh_list()
448         if not w_list then return end
449         w_list:clear()
450         for _, r in ipairs(rows) do
451             local label = string.format("#%d  %s -> %s  [%s, %s]", r.n, fmt_clock(r.s),
fmt_clock(r.e), r.reason, r.sport)
452             if r.shots ~= nil then label = label .. string.format("  %s shots",

```

```

tostring(r.shots)) end
453     w_list:add_value(label, r.n)
454     end
455     if d then d:update() end
456 end
457
458 -- Action-button labels reflect state. Save shows the rally number it will write
459 -- (or "Save changes (#N)" while editing). Undo last shows exactly which row it
460 -- will remove ("Undo last (#N)"), or that it will cancel an edit / clear a mark.
461 local function refresh_buttons()
462     if w_save then
463         if mode == "edit" and edit_index and rows[edit_index] then
464             w_save:set_text(string.format("Save changes (##d)", rows[edit_index].n))
465         else
466             local s = get_field_num(w_start)
467             local e = get_field_num(w_end)
468             if s and e then
469                 w_save:set_text(string.format("Save Rally (##d)", planned_next_number()))
470             else
471                 w_save:set_text("Save Rally")
472             end
473         end
474     end
475     -- While editing, the Mark buttons re-mark the EDITED rally's times (not a new one),
476     -- so relabel them to make that unmissable -- a forgotten edit-mode is how you'd
477     -- accidentally overwrite an existing rally thinking you were creating a new one.
478     if w_mark_start and w_mark_end then
479         if mode == "edit" and edit_index and rows[edit_index] then
480             w_mark_start:set_text(string.format("Re-mark START (##d)", rows[edit_index].n))
481             w_mark_end:set_text(string.format("Re-mark END (##d)", rows[edit_index].n))
482         else
483             w_mark_start:set_text("Mark START")
484             w_mark_end:set_text("Mark END")
485         end
486     end
487     if w_undo then
488         if mode == "edit" then
489             w_undo:set_text("Undo last (cancel edit)")
490         else
491             local s = get_field_num(w_start)
492             local e = get_field_num(w_end)
493             if s or e then
494                 w_undo:set_text("Undo last (clear mark)")
495             elseif #rows > 0 then
496                 w_undo:set_text(string.format("Undo last (##d)", rows[#rows].n))
497             else
498                 w_undo:set_text("Undo last")
499             end
500         end
501     end
502     if d then d:update() end
503 end
504
505 local function set_status(msg)
506     if not w_status then return end
507     local mode_line
508     if mode == "edit" and edit_index and rows[edit_index] then
509         mode_line = string.format(
510             "Mode: EDITING rally ##d (Save changes to commit, Undo last to cancel).",
511             rows[edit_index].n)
512     else
513         mode_line = "Mode: new rally (Mark START, Mark END, choose reason, Save Rally)."
514     end
515     local last_line = ""
516     if #rows > 0 then

```



```

516         local lr = rows[#rows]
517         last_line = string.format("Last row (what Undo removes): #%d %s -&gt; %s
[%s]<br>",
518             lr.n, esc(fmt_clock(lr.s)), esc(fmt_clock(lr.e)), esc(lr.reason))
519     end
520     w_status:set_text(string.format(
521         "%s<br>%s<br>%sNow: %s &nbsp;&nbsp;|&nbsp;   Rallies in CSV: %d<br>CSV: %s",
522         esc(msg), esc(mode_line), last_line, esc(fmt_clock(now_seconds()))), #rows,
esc(out_path)))
523     if d then d:update() end
524 end
525
526 -- Clear the form back to a fresh "new rally" state (reason reset = non-sticky).
527 local function reset_form()
528     mode = "new"
529     edit_index = nil
530     if w_start then w_start:set_text("") end
531     if w_end then w_end:set_text("") end
532     if w_shots then w_shots:set_text("") end
533     rebuild_reason_default()
534     refresh_buttons()
535     if d then d:update() end
536 end
537
538 -- Re-resolve the CSV path against whatever video is playing NOW and load its
539 -- labels. Lets the user open a video later (or switch videos) and pick up exactly
540 -- where they left off without toggling the extension off/on. Same video -> just
541 -- re-read the CSV; a different video -> re-point and load that video's rallies.
542 local function sync_to_current_video()
543     if mode == "edit" then
544         -- Reloading rows would invalidate edit_index; make the user resolve the edit first.
545         refresh_list(); refresh_buttons()
546         set_status("Finish (Save changes) or cancel (Undo last) the current edit before
refreshing.")
547         return
548     end
549     local media = current_media_path()
550     if not media then
551         refresh_list(); refresh_buttons()
552         set_status("No media is playing -- still using this CSV. Open the video, then click
Refresh.")
553         return
554     end
555     out_path_is_fallback = false -- a video is playing now -> we have a real path
556     local new_path = resolve_out_path()
557     if new_path == out_path then
558         load_rows() -- re-read in case the file changed on disk
559         refresh_next_field()
560         refresh_list(); refresh_buttons()
561         set_status(string.format("Refreshed. %d rallies loaded for this video.", #rows))
562         return
563     end
564     out_path = new_path -- switched videos -> re-point + resume that file
565     load_rows()
566     reset_form()
567     refresh_next_field()
568     refresh_list()
569     refresh_buttons()
570     set_status(string.format(
571         "Switched to the current video. Loaded %d existing rallies; next is #%d.", #rows,
next_rally_number()))
572 end
573
574 -- Lazily adopt the playing video's CSV. We may have started on the home-dir FALLBACK
575 -- (extension enabled before the video was loaded, so no path was resolvable then). The

```

```

576     -- moment a timestamp exists the video is provably playing, so switch to
<video>.rallies.csv
577     -- and load that video's existing rallies BEFORE anything is written to the fallback --
this
578     -- is what makes "enable, then open the video" and pause/restart resume correctly
instead of
579     -- silently writing to ~/rally_labels.csv. No-op once we're already on a real video CSV.
580     local function adopt_current_video_csv()
581         if not out_path_is_fallback then return end
582         local media = current_media_path()
583         if not media then return end             -- still no video -> stay on the fallback for
now
584         out_path_is_fallback = false             -- a real path is resolvable now
585         local p = resolve_out_path()
586         if p == out_path then return end         -- already pointing there (defensive)
587         out_path = p
588         load_rows()                             -- pick up rallies already saved for this video
589         refresh_next_field()
590         refresh_list()
591     end
592
593     -- Seek the player by delta_s seconds (relative), clamped at 0 (time is microseconds in
3.x).
594     local function seek_by(delta_s)
595         local input = vlc.object.input()
596         if not input then set_status("No media playing -- cannot seek."); return end
597         local t = vlc.var.get(input, "time")     -- microseconds
598         if not t then set_status("No playback time available to seek from."); return end
599         local nt = t + delta_s * 1000000
600         if nt < 0 then nt = 0 end
601         vlc.var.set(input, "time", nt)
602         set_status(string.format("Seek %+ds -> %s.", delta_s, fmt_clock(nt / 1000000.0)))
603     end
604
605     -----
606     -- Button callbacks (VLC calls these on its main loop; kept global to be safe)
607     -----
608     function mark_start()
609         local t = now_seconds()
610         if not t then set_status("No media playing -- cannot mark START."); return end
611         if is_armed() then
612             set_status("You have an UNSAVED rally (START -> END). Click 'Save Rally' to keep it,
or 'Undo last' to "
613                 .. "clear it, before marking a new START. (You can also edit the Start field by
hand.)")
614             return
615         end
616         adopt_current_video_csv()    -- video is playing now: make sure we're on its CSV, not
the fallback
617         w_start:set_text(string.format("%.3f", t))
618         refresh_buttons()
619         set_status(string.format("START set @ %s. Play to the rally's end, then Mark END.",
fmt_clock(t)))
620     end
621
622     function mark_end()
623         local t = now_seconds()
624         if not t then set_status("No media playing -- cannot mark END."); return end
625         w_end:set_text(string.format("%.3f", t))
626         refresh_buttons()
627         set_status(string.format("END set @ %s. Choose the Ending reason, then click Save
Rally.", fmt_clock(t)))
628     end
629
630     function save_rally()

```

```

631     adopt_current_video_csv() -- in case times were typed by hand without Mark START
632     local s = get_field_num(w_start)
633     local e = get_field_num(w_end)
634     if not s then set_status("Set a START time first (click Mark START)."); return end
635     if not e then set_status("Set an END time first (click Mark END)."); return end
636     if e < s then s, e = e, s end -- tolerate reversed marks
637     if e <= s then
638         set_status("END must be later than START (rally must be > 0s).")
639         return
640     end
641     local _, reason = w_reason:get_value() -- (id, text); "unknown" is allowed
642     if not reason or reason == "" then reason = REASON_DEFAULT end
643     local _, sport = w_sport:get_value()
644     if not sport or sport == "" then sport = SPORTS[1] end
645     local shots = get_shots() -- optional; nil when blank
646     local shots_note = (shots ~= nil) and string.format(", %d shots", shots) or ""
647
648     local msg
649     if mode == "edit" and edit_index and rows[edit_index] then
650         local r = rows[edit_index]
651         r.s, r.e, r.reason, r.sport, r.shots = s, e, reason, sport, shots
652         local ok, err = save_all()
653         if not ok then set_status("WRITE FAILED: " .. tostring(err)); return end
654         msg = string.format("Updated rally #%d: %s -> %s [%s, %s%s].", r.n, fmt_clock(s),
fmt_clock(e), reason, sport, shots_note)
655     else
656         local n = planned_next_number()
657         if index_of_rally(n) then
658             set_status(string.format("Rally #%d already exists -- set \"Next rally #\" to a
free number.", n))
659             return
660         end
661         rows[#rows + 1] = { n = n, s = s, e = e, reason = reason, sport = sport, shots =
shots }
662         local ok, err = save_all()
663         if not ok then rows[#rows] = nil; set_status("WRITE FAILED: " .. tostring(err));
return end
664         if w_next then w_next:set_text(tostring(next_free_from(n + 1))) end -- next free
number
665         msg = string.format("Saved rally #%d: %s -> %s [%s, %s%s].", n, fmt_clock(s),
fmt_clock(e), reason, sport, shots_note)
666     end
667
668     reset_form() -- clears fields + resets the (required) reason to placeholder
669     refresh_list()
670     set_status(msg)
671 end
672
673 function edit_selected()
674     if is_armed() then
675         set_status("Finish the current rally ('Save Rally') or clear it ('Undo last') before
editing another "
676             .. "-- your unsaved START -> END would be lost.")
677         return
678     end
679     local n = selected_rally_number()
680     if not n then set_status("Pick a rally in the Recent list first, then Edit
selected."); return end
681     local idx = index_of_rally(n)
682     if not idx then set_status("Rally #" .. tostring(n) .. " not found (try Refresh).");
return end
683     local r = rows[idx]
684     mode = "edit"; edit_index = idx
685     w_start:set_text(string.format("%.3f", r.s))
686     w_end:set_text(string.format("%.3f", r.e))

```

```

687     if w_shots then w_shots:set_text(r.shots ~= nil and tostring(r.shots) or "") end
688     rebuild_reason_selected(r.reason)
689     rebuild_sport_selected(r.sport)
690     refresh_buttons()
691     set_status(string.format(
692         "Editing rally #%. Adjust Start/End/reason, then Save changes. (Undo last
cancels.)", r.n))
693     end
694
695     function delete_selected()
696         local n = selected_rally_number()
697         if not n then set_status("Pick a rally in the Recent list first, then Delete
selected."); return end
698         local idx = index_of_rally(n)
699         if not idx then set_status("Rally #" .. tostring(n) .. " not found (try Refresh).");
return end
700         table.remove(rows, idx)
701         local ok, err = save_all()
702         if not ok then set_status("WRITE FAILED: " .. tostring(err)); return end
703         reset_form()
704         refresh_next_field()
705         refresh_list()
706         set_status(string.format("Deleted rally #%. %d remaining.", n, #rows))
707     end
708
709     function undo_last()
710         if mode == "edit" then
711             reset_form(); refresh_list(); set_status("Edit cancelled.")
712             return
713         end
714         local s = get_field_num(w_start)
715         local e = get_field_num(w_end)
716         if s or e then
717             reset_form()
718             set_status("Cleared the in-progress START/END (nothing was written).")
719             return
720         end
721         if #rows == 0 then set_status("Nothing to undo."); return end
722         local last = rows[#rows]
723         rows[#rows] = nil
724         local ok, err = save_all()
725         if not ok then rows[#rows + 1] = last; set_status("WRITE FAILED: " .. tostring(err));
return end
726         refresh_list()
727         refresh_next_field()
728         refresh_buttons()
729         set_status(string.format("Removed last rally #%. %d remaining.", last.n, #rows))
730     end
731
732     function refresh_now()
733         sync_to_current_video()
734     end
735
736     -- Playback control (verified available to VLC 3.x extensions). pause() is a TOGGLE
737     -- (playlist_TogglePause), so we branch on status() to keep one Play / Pause button
738     -- deterministic: pause() flips playing<->paused, and from stopped we play() fresh.
739     function play_pause()
740         local st = vlc.playlist.status()
741         if st == "playing" then
742             vlc.playlist.pause() -- toggles playing -> paused
743             set_status("Paused. Annotate (or fine-tune Start/End), then Play / Pause to
resume.")
744         elseif st == "paused" then
745             vlc.playlist.pause() -- toggles paused -> playing
746             set_status("Resumed playback.")

```

```

747     elseif st == "stopped" then
748         vlc.playlist.play()
749         set_status("Started playback.")
750     else
751         set_status("No media loaded to play.")
752     end
753 end
754
755 function seek_back() seek_by(-5) end
756 function seek_fwd() seek_by(5) end
757
758 function show_help()
759     -- Toggle a DEDICATED help panel by adding/removing a widget. In-place set_text on
760     -- the shared status widget did not repaint reliably in VLC (the panel got "stuck");
761     -- add_html + del_widget forces a real layout change that VLC renders every time.
762     if w_help then
763         if d then d:del_widget(w_help) end
764         w_help = nil
765         if w_help_btn then w_help_btn:set_text("Help") end
766     else
767         if d then w_help = d:add_html(HELP_HTML, 1, 16, 4, 8) end
768         if w_help_btn then w_help_btn:set_text("Hide help") end
769     end
770     if d then d:update() end
771 end
772
773 -----
774 -- Dialog construction
775 -----
776 local function create_dialog()
777     d = vlc.dialog("Rally Annotator v" .. VERSION)
778
779     d:add_label("Sport:", 1, 1, 1, 1)
780     w_sport = d:add_dropdown(2, 1, 2, 1)
781     for i, v in ipairs(SPORTS) do w_sport:add_value(v, i) end    -- badminton first =>
default
782     w_help_btn = d:add_button("Help", show_help, 4, 1, 1, 1)
783
784     -- Playback controls -- one row of 3: Back 5s | Play / Pause (toggle) | Fwd 5s.
785     d:add_button("Back 5s", seek_back, 1, 2, 1, 1)
786     d:add_button("Play / Pause", play_pause, 2, 2, 2, 1)    -- spans cols 2-3, centered
787     d:add_button("Fwd 5s", seek_fwd, 4, 2, 1, 1)
788
789     d:add_label("Start (s):", 1, 3, 1, 1)
790     w_start = d:add_text_input("", 2, 3, 1, 1)
791     d:add_label("End (s):", 3, 3, 1, 1)
792     w_end = d:add_text_input("", 4, 3, 1, 1)
793
794     d:add_label("Next rally #:", 1, 4, 1, 1)
795     w_next = d:add_text_input("", 2, 4, 1, 1)
796     d:add_label("Number of shots:", 3, 4, 1, 1)    -- optional; blank => shots_count left
empty
797     w_shots = d:add_text_input("", 4, 4, 1, 1)
798
799     d:add_label("Ending reason:", 3, 5, 1, 1)    -- labels the reason dropdown directly
below it
800
801     -- Per-rally commit row, left-to-right: Mark START -> Mark END -> reason -> Save.
802     w_mark_start = d:add_button("Mark START", mark_start, 1, 6, 1, 1)
803     w_mark_end = d:add_button("Mark END", mark_end, 2, 6, 1, 1)
804     rebuild_reason_default()    -- creates w_reason at (3,6,1,1), under
its label
805     w_save = d:add_button("Save Rally", save_rally, 4, 6, 1, 1)
806
807     w_status = d:add_html("", 1, 7, 4, 2)    -- rich-text status panel (multi-line via

```

```

<br>)
808
809     d:add_label("Recent rallies (select one, then Edit/Delete):", 1, 9, 4, 1)
810     w_list = d:add_list(1, 10, 4, 4)
811
812     d:add_button("Edit selected",    edit_selected,    1, 14, 1, 1)
813     d:add_button("Delete selected",  delete_selected, 2, 14, 1, 1)
814     w_undo = d:add_button("Undo last", undo_last,      3, 14, 1, 1)
815     d:add_button("Refresh",          refresh_now,      4, 14, 1, 1)
816
817     refresh_next_field()
818     refresh_list()
819     refresh_buttons()
820     d:show()
821     if out_path_is_fallback then
822         set_status("No video detected yet. Open/play the video, then click Mark START -- the
tool will switch to "
823             .. "that video's own .rallies.csv automatically (and load any rallies already
saved for it). "
824             .. "Tip: open the video FIRST, then enable this extension.")
825     elseif #rows > 0 then
826         set_status(string.format(
827             "Resumed this video: %d existing rallies loaded (next is #%d). Click Help for the
guide.",
828             #rows, next_rally_number()))
829     else
830         set_status("Ready. Pick the Sport, then mark rallies. Click Help for usage + the
ending-reason guide.")
831     end
832 end
833
834 -----
835 -- Lifecycle
836 -----
837 function activate()
838     -- Build the id lookups here (full Lua environment), not at top level (see note
839     -- where REASON_ID/SPORT_ID are declared -- the descriptor scan sandbox lacks ipairs).
840     for i, v in ipairs(REASONS) do REASON_ID[v] = i end
841     for i, v in ipairs(SPORTS) do SPORT_ID[v] = i end
842     mode = "new"
843     edit_index = nil
844     w_help = nil
845     w_reason = nil          -- niled so the first rebuild on a re-enable doesn't del a
stale handle
846     out_path = resolve_out_path()
847     out_path_is_fallback = (current_media_path() == nil)  -- no video yet -> adopt its
CSV lazily on first Mark
848     load_rows()            -- continue an existing CSV (numbering + recent list)
849     create_dialog()
850 end
851
852 function deactivate()
853     if d then d:delete(); d = nil end
854     w_help = nil
855 end
856
857 function close()
858     deactivate()
859     vlc.deactivate()
860 end
861

```

15. user (2026-06-20T10:03:21.937Z)

NO_IMAGE_FILES_TRACKED

16. assistant (2026-06-20T10:03:26.773Z)

I have the full source. No image files are tracked. Let me check the test README (for the dialog layout snapshot, which is the screenshot stand-in) and verify there are no untracked images anywhere in the working tree (excluding .git), to be thorough about screenshots.

17. user (2026-06-20T10:03:27.716Z)

```
1      # Tests
2
3      The extension (`vlc/rally_annotator.lua`) is a single Lua file that only talks to VLC
4      through the global `vlc` table. `dialog_test.lua` **stubs that table** (dialog widgets +
5      playback) in pure Lua, loads the real extension, drives its callbacks, and asserts the
6      resulting state.
7
8      ## Run
9
10     Use **Lua 5.1** ? the same interpreter VLC 3.x embeds, so the syntax/semantics match:
11
12     ```bash
13     lua5.1 test/dialog_test.lua          # from the repo root
14     ```
15
16     Exit code is `0` if all assertions pass, `1` otherwise ? so it drops straight into CI or
17     a
18     pre-commit hook. **GitHub Actions runs it on every push and PR**
19     ([`.github/workflows/ci.yml`](../.github/workflows/ci.yml)):
20     it syntax-checks the extension with `luac5.1 -p`, then runs this suite (logic **the
21     layout snapshot**).
22
23     ### Installing Lua 5.1
24
25     - **Debian/Ubuntu (incl. WSL):** `sudo apt install -y lua5.1`
26     - **macOS:** `brew install lua@5.1`
27     - **Windows ? no admin, portable (recommended):** download the official LuaBinaries
28     5.1.5 build and unzip it into
29     your user folder. PowerShell:
30
31     ```powershell
32     $dir = "$env:USERPROFILE\tools\lua515"
33     New-Item -ItemType Directory -Force $dir | Out-Null
34     $zip = "$env:TEMP\lua515.zip"
35     # Use the direct file host (the SourceForge "/download" page is HTML, not the zip):
36     Invoke-WebRequest "https://master.dl.sourceforge.net/project/luabinaries/5.1.5/Tools%2
37     0Executables/lua-5.1.5_Win64_bin.zip?viasf=1" -OutFile $zip -UseBasicParsing
38     Expand-Archive $zip -DestinationPath $dir -Force
39     & "$dir\lua5.1.exe" -v          # -> Lua 5.1.5
40     & "$dir\lua5.1.exe" test\dialog_test.lua # run the suite (from repo root)
41     ```
42
43     Add ``$env:USERPROFILE\tools\lua515`` to your PATH to call `lua5.1` directly.
44     - **Windows ? with admin:** in an **elevated** PowerShell (Chocolatey needs admin, so a
45     normal shell fails with
46     `Access to the path ...chocolatey... is denied`): `choco install lua51 -y`.
47
48     Why 5.1 specifically: it's the interpreter VLC 3.x embeds, so the test's
49     syntax/semantics match what ships in VLC.
50
51     ## What it covers
52
53     - **Load safety** ? `descriptor()` returns valid metadata and the version is current.
54     - **Window title** ? the dialog title carries the version and stays exactly in sync with
55     `descriptor().version`.
56     - **Layout snapshot** ? the full widget grid (every control's kind, grid column/row,
57     column/row span, and
```

```

49     caption/options) plus the window title are serialized and diffed against a committed
golden,
50     `dialog_layout.snapshot` (see below).
51     - Dialog construction ? every control (playback, Mark START/END, Save, reason
dropdown, Number of shots input) is wired to its callback / present at its grid cell.
52     - Help toggle ? the dedicated help panel is added/removed on each click and the
button label flips.
53     - Reason field ? defaults to `unknown`, resets after every save, supports the same
reason on consecutive rallies, and a no-pick save records `unknown`.
54     - Shots field ? the optional `shots_count` is written when entered, left blank when
not, clears after each save, and is reloaded by Edit selected (and the edit rewrites it).
55     - Recent list ? loading a 13-rally CSV lists all of them (oldest first, newest
last), with no "last N" cap hiding the earliest rallies.
56     - Resume / CSV adoption ? enabling with no media (home fallback) then "playing" a
video and clicking Mark START adopts `Unsaved-work guards ? with a fully-marked-but-unsaved rally (START + END), Mark
START and Edit selected are refused (fields untouched) until you Save Rally or Undo last;
in edit mode the Mark buttons relabel to "Re-mark START/END (#N)" and revert afterwards.
58     - Numbering ? the "Next rally #" field auto-advances to the next free number,
refuses a duplicate, and re-syncs after Undo so removing a rally leaves no gap.
59     - CSV output ? the bytes actually written are checked (6-column header + rows + the
shots column + that an undone rally is absent).
60     - Playback ? the single Play / Pause toggle branches on `vlc.playlist.status()`
so it never flips the wrong way (and starts fresh from stopped), and seek does relative ± with a
clamp at 0.
61
62     ## Layout snapshot (`dialog_layout.snapshot`)
63
64     VLC renders extension dialogs through its Qt GUI and has no headless/offscreen
rendering path, and no CLI
65     flag to auto-open a Lua extension ? so a real pixel screenshot can't be captured in an
automated, deterministic,
66     cross-platform way (it would need a virtual display + scripted menu clicks + flaky pixel
diffs). Instead the test
67     captures the widget tree the extension hands to VLC ? the same thing VLC turns into
pixels ? and diffs it
68     against a committed golden:
69
70     ```
71     title :: Rally Annotator v1.6
72     1. label      @(1,1) 1x1 :: Sport:
73     2. dropdown  @(2,1) 2x1 :: [badminton, tennis, table_tennis, pickleball, padel]
74     ...
75     18. dropdown @(3,6) 1x1 :: [unknown, winner, forced_error, unforced_error,
service_fault, let, other]
76     ```
77
78     Each line is a control's `kind @(column,row) WxH :: caption` (dropdowns list their
options; the dynamic status/help
79     HTML and the recent-rallies list are shown as placeholders since their bodies are
runtime data, not layout). It's
80     captured right after `activate()`, in the dialog's pristine initial state. A moved,
resized, renamed, added, or
81     removed control ? or a changed window title ? fails the test with a line-by-line
`want`/`got` diff.
82
83     When you change the layout on purpose, regenerate the golden and commit it:
84
85     ```bash
86     lua5.1 test/dialog_test.lua --update      # or: UPDATE_SNAPSHOT=1 lua5.1
test/dialog_test.lua
87     ```
88
89     ## What it can't cover

```



```
90
91     It exercises all the **logic** and the **declared layout**, but not VLC's actual GUI
92     *rendering* (real fonts, Qt
93     theming, DPI) or real media playback ? there's no headless VLC UI automation. Those are
94     validated separately by
95     loading the extension in VLC with `-vv --file-logging` (it must scan with no `Error
96     loading` line) and a manual
97     click-through. See the repo root `README.md` for that flow.
```

18. user (2026-06-20T10:03:28.275Z)

```
1      title :: Rally Annotator v1.6.4
2      1. label      @(1,1) 1x1 :: Sport:
3      2. dropdown   @(2,1) 2x1 :: [badminton, tennis, table_tennis, pickleball, padel]
4      3. button      @(4,1) 1x1 :: Help
5      4. button      @(1,2) 1x1 :: Back 5s
6      5. button      @(2,2) 2x1 :: Play / Pause
7      6. button      @(4,2) 1x1 :: Fwd 5s
8      7. label      @(1,3) 1x1 :: Start (s):
9      8. text_input  @(2,3) 1x1 :: ""
10     9. label      @(3,3) 1x1 :: End (s):
11     10. text_input @(4,3) 1x1 :: ""
12     11. label      @(1,4) 1x1 :: Next rally #:
13     12. text_input @(2,4) 1x1 :: "1"
14     13. label      @(3,4) 1x1 :: Number of shots:
15     14. text_input @(4,4) 1x1 :: ""
16     15. label      @(3,5) 1x1 :: Ending reason:
17     16. button      @(1,6) 1x1 :: Mark START
18     17. button      @(2,6) 1x1 :: Mark END
19     18. dropdown   @(3,6) 1x1 :: [unknown, winner, forced_error, unforced_error,
20     service_fault, let, other]
21     19. button      @(4,6) 1x1 :: Save Rally
22     20. html        @(1,7) 4x2 :: <runtime html>
23     21. label      @(1,9) 4x1 :: Recent rallies (select one, then Edit/Delete):
24     22. list        @(1,10) 4x4 :: <list>
25     23. button      @(1,14) 1x1 :: Edit selected
26     24. button      @(2,14) 1x1 :: Delete selected
27     25. button      @(3,14) 1x1 :: Undo last
28     26. button      @(4,14) 1x1 :: Refresh
```

19. user (2026-06-20T10:03:30.119Z)

---UNTRACKED---

20. assistant (2026-06-20T10:03:36.243Z)

I now have complete coverage of the repo. No image/screenshot files exist anywhere in the working tree (tracked or untracked) ? the ``dialog_layout.snapshot`` is the deterministic text-based stand-in for a screenshot. The working tree is clean with no untracked files.

Let me verify the latest version tag/release state to confirm "latest version" precisely.

21. user (2026-06-20T10:03:38.703Z)

```
---TAGS---
---REMOTE---
origin https://github.com/avidullu/rally-annotator.git (fetch)
origin https://github.com/avidullu/rally-annotator.git (push)
---HEAD DESCRIBE---
no tags
```

22. assistant (2026-06-20T10:03:42.880Z)

No git tags; versioning is via the `VERSION` constant and CHANGELOG. Latest is v1.6.4 (confirmed in three independent places: `vlc/rally\_annotator.lua:88`, the dialog title snapshot, and `CHANGELOG.md:3`). I have everything I need. Let me compose the structured output.

23. user (2026-06-20T10:04:58.865Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 15409 bytes): {"what_it_is": "[verified] rally-annotator is a single-file VLC Lua **extension** (not a standalone app) that lets a human hand-label rally start/end + a point-ending reason while watching a match in
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

24. assistant (2026-06-20T10:05:01.233Z)

The input got wrapped in a `raw` envelope and was malformed. Let me call StructuredOutput with proper structured fields directly.

25. user (2026-06-20T10:06:18.527Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 16128 bytes): {"what_it_is": "[verified] rally-annotator is a single-file VLC Lua EXTENSION (not a standalone app) that lets a human hand-label rally start/end + a point-ending reason while watching a match in VLC,
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

26. assistant (2026-06-20T10:06:21.978Z)

The harness keeps wrapping my input in a `raw` key. I need to pass the actual structured object fields, not a JSON string. Let me provide each field as a plain string value with simpler content and avoid problematic escaping.

27. user (2026-06-20T10:07:32.929Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 15392 bytes): {"what_it_is": "[verified] rally-annotator is a single-file VLC Lua EXTENSION (not a standalone app) that lets a human hand-label rally start/end + a point-ending reason while watching a match in VLC,
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

28. assistant (2026-06-20T10:07:37.605Z)

The issue is the unicode em-dash character (`\u2014`) and possibly the size. The harness shows my input is being received under a `raw` wrapper which suggests a serialization mismatch. Let me retry with plain ASCII only (no em-dashes, no smart quotes, no unicode escapes), shorter content, and carefully escaped quotes/newlines.

29. user (2026-06-20T10:08:45.202Z)

Output does not match required schema: root: must have required property 'what_it_is', root: must have required property 'install', root: must have required property 'annotate_workflow', root: must have required property 'output_csv', root: must have required property 'screenshots', root: must have required property 'repo_facts', root: must have required property 'file_refs', root: must NOT have additional properties

30. assistant (2026-06-20T10:08:49.350Z)

The harness is wrapping my arguments under a top-level key. The error shows the validator sees

my whole object as an additional property. This means my JSON is being placed inside a wrapper. Let me try passing the arguments as a normal tool-call object (the standard way), with the backslashes in Windows paths properly escaped.

31. user (2026-06-20T10:09:48.130Z)

Structured output provided successfully

32. assistant (2026-06-20T10:09:59.013Z)

Structured output delivered successfully. Summary of what I found in the rally-annotator repo (C:\Users\avidu\Projects\rally-annotator, branch main, HEAD d5d59f2):

- **What it is**: a single-file VLC Lua extension (`\vlc\rally_annotator.lua`, v1.6.4) for hand-labeling rally start/end + point-ending reason across 5 net-separated racquet sports, producing one CSV row per rally. It's the literal "annotate your footage -> golden labels" entry point and originated inside badminton-highlight-indexer.
- **Install**: copy one .lua file into VLC's per-user `\lua\extensions\` folder (Win/macOS/Linux paths verified), Reload extensions, enable from View menu. VLC 3.0.x required; no build/deps.
- **Workflow**: click-driven (NO marking hotkeys today ? hotkeys are roadmap only). Mark START -> Mark END (arms) -> pick reason + optional shots -> Save Rally. Built-in playback, non-sticky reason reset, resumable numbering, atomic writes, unsaved-work guard.
- **Output CSV schema**: `\rally_number,start_time,end_time,ending_reason,sport,shots_count` (decimal seconds; shots_count optional/blank), written to `<video-stem>.rallies.csv` next to the video (home-dir `\rally_labels.csv` fallback).
- **Screenshots**: NONE exist anywhere in the repo (tracked or untracked) ? confirmed. The text `\test\dialog_layout.snapshot` is the deterministic stand-in for a screenshot. Studio UI screenshots will need to be captured by loading the extension in VLC.
- **Repo facts**: MIT license; latest v1.6.4 (no git tags, version is the VERSION constant + CHANGELOG); cross-platform; CI (luac syntax + 65-assertion Lua 5.1 suite).

I flagged honestly in the design notes that the annotator is real/shipping but the downstream "train your own model" loop in the indexer is largely roadmap/owner-gated, so Studio pages must not imply one-click training exists today.